

# Compositional neurosymbolic representations enable efficient active exploration

Received: 29 April 2024

Accepted: 9 July 2026

Published online: 22 August 2026

 Check for updatesP. Michael Furlong<sup>1,2,3,8</sup>✉, Nicole S.-Y. Dumont<sup>4,8</sup>, Rika Antonova<sup>5,8</sup>, Jeff Orchard<sup>1,6</sup> & Chris Eliasmith<sup>1,2,7</sup>

Autonomous systems that learn and explore over long horizons face a problem. Standard methods scale poorly in the number of observations,  $n$ , precluding sustained operation on bounded hardware. We show that compositional, high-dimensional vector representations inspired by neural computation address these constraints. We use these representations to construct a Bayesian optimization (BO) algorithm that operates in complex spaces and reduces the time and memory requirements compared to state-of-the-art BO algorithms on diverse tasks. Whereas standard methods incur  $O(n^3)$  time and  $O(n^2)$  memory complexity, our approach holds both at  $O(d^2)$  in the embedding dimension, which remains constant over the algorithm's lifetime. Our algorithm reduces compute time by 60–200 × without loss in accuracy. Implementation on neuromorphic hardware reduces energy consumption per sample by 30–188 ×. These efficiencies stem from converting sample selection into continuous optimization on a compact domain, implementable by gradient methods or neural dynamics, enabling long-term, resource-bound, autonomous exploration.

Generalizing efficiently from limited data in complex environments remains a major challenge for artificial intelligence (AI). Humans, in contrast, learn and act in complex, high-dimensional state and action spaces with remarkable efficiency, even in the face of uncertainty. A crucial aspect of this ability is the use of compositional representations, where simpler elements are systematically combined to form more complex structures. This principle underlies human cognition, from perception to language, and is fundamental to generalization and adaptability in complex environments<sup>1–4</sup>. However, achieving both *scalability* and *compositional* expressiveness in AI models remains an open challenge<sup>5,6</sup>.

In machine learning, kernel methods offer a principled way to encode structured inductive biases<sup>7</sup>, support data-efficient learning,

and model uncertainty. Compositional kernels, constructed by combining simpler kernels via addition or multiplication, enable the expression of rich relational structures<sup>8</sup>. However, the practical application of such kernels is hindered by scalability: inference with Gaussian processes (GPs) requires  $O(n^3)$  time and  $O(n^2)$  memory, where  $n$  is the number of observations. While kernel approximation methods such as Random Fourier Features [RFFs; <sup>9</sup>] reduce this complexity, approximating compositional kernels with RFFs often leads to unbounded growth in embedding dimensionality<sup>10</sup>, and representing structured or symbolic inputs remains challenging.

Vector Symbolic Algebras (VSAs) - specifically, Holographic Reduced Representations (HRRs) and their complex-valued variant, Fourier HRRs (FHRRs)<sup>11,12</sup>—offer a promising solution by providing a

<sup>1</sup>Centre for Theoretical Neuroscience, University of Waterloo, Waterloo, Canada. <sup>2</sup>Department of Systems Design Engineering, University of Waterloo, Waterloo, Canada. <sup>3</sup>National Research Council Canada, Ottawa, Canada. <sup>4</sup>Institute of Neuroinformatics, University of Zurich, Rämistrasse 71, Zurich, Switzerland. <sup>5</sup>University of Cambridge, Cambridge, United Kingdom. <sup>6</sup>Cheriton School of Computer Science, University of Waterloo, Waterloo, Canada. <sup>7</sup>Department of Philosophy, University of Waterloo, Waterloo, Canada. <sup>8</sup>These authors contributed equally: P. Michael Furlong, Nicole S.-Y. Dumont, Rika Antonova. ✉e-mail: [michael.furlong@uwaterloo.ca](mailto:michael.furlong@uwaterloo.ca)

dimensionality-preserving algebra for compositional representation. We note that VSAs are more commonly known as *vector symbolic architectures*<sup>13</sup>, but we use the term algebra to distinguish the algebras from the systems architected using VSAs.

Vector embeddings used in kernel approximation (such as RFFs) typically implement composition using operations like concatenation (for additive kernels) or tensor products (for multiplicative kernels), both of which increase dimensionality. In contrast, VSAs achieve similar compositionality using fixed-dimensional operations such as bundling and binding (e.g., circular convolution in HRRs). This allows vector embeddings of structured and hierarchical data to have fixed dimensionality.

VSAs have a long history in cognitive science<sup>6,13–21</sup>, and have more recently been used for neural-symbolic deep learning<sup>21,22</sup>, differentiable memory<sup>23</sup>, efficient attention<sup>24,25</sup>, and kernel approximation.

Indeed, the connection between VSAs and kernel embeddings has been well understood since their inception<sup>26</sup>, but has garnered recent attention<sup>27–31</sup>. The central hypothesis of VSAs is that cognition operates through algebraic manipulations of high-dimensional distributed representations. These operations support not only compositionality and neural implementation<sup>20</sup>, but also the encoding of uncertainty<sup>29,30,32</sup>.

In this work, we introduce Spatial Semantic Pointer Bayesian optimization (SSP-BO), a new Bayesian optimization (BO) algorithm based on VSA-derived kernel approximations. Our method leverages the algebra of HRRs to construct expressive, compositional, and fixed-size embeddings over various action spaces. Additionally, our work further connects BO to biologically plausible neural representations through the use of continuous vector encodings called Spatial Semantic Pointers (SSPs), also referred to as fractional power encodings<sup>33–35</sup>. These representations reproduce activity patterns observed in the brain's spatial coding systems—such as grid, place, and head direction cells<sup>36–38</sup>—and serve as our algorithm's embedding method for continuous variables.

This yields several advantages:

- **Scalability:** We obtain constant-time sample selection with respect to the number of environment observations,  $n$ . Standard BO requires  $\mathcal{O}(n^3)$  time and  $\mathcal{O}(n^2)$  memory for surrogate model updates; our approach reduces this to  $\mathcal{O}(d^2)$  and  $\mathcal{O}(d^2)$ , respectively, where  $d$  is the fixed dimensionality of the VSA embeddings. Since  $d$  is set prior to optimization and remains constant, both sample generation time and memory usage are independent of the number of observations collected during the run, and hence  $\mathcal{O}(1)$  with respect to  $n$ .
- **Expressivity:** Our method supports structured, symbolic, and mixed discrete-continuous input spaces (e.g., graphs, sets, trajectories) via a single algebraic framework.
- **Neuromorphic compatibility:** Sample selection in SSP-BO occurs in the continuous embedding space. This makes it possible to perform sample selection over structured action spaces on neuromorphic hardware, providing a biologically grounded pathway for efficient decision-making systems.

We evaluate SSP-BO on a suite of tasks involving action selection in structured domains, including robotic trajectory optimization, multi-agent coordination, graph-based search, and combinatorial design. Compared to exact GP-based methods, SSP-BO offers substantial efficiency gains while maintaining strong performance. Whereas existing approaches typically provide either scalable BO or expressive BO, this work combines both within a single compositional kernel embedding framework, enabling efficient optimization in structured domains and supporting neuromorphic implementation. To the best of our knowledge, this constitutes the first demonstration of neuromorphic hardware being used within a BO loop. A graphical abstract is provided in Fig. 1.

Bayesian optimization (BO) is a data-efficient framework for black-box optimization and reinforcement learning<sup>39–41</sup>. It has been applied to problems in autonomous exploration, morphology and material design, and hyperparameter tuning<sup>42–47</sup>. BO typically models an unknown reward or objective function with a surrogate (often a GP [refs. 43,48–50]) and selects queries by optimizing an acquisition function over the surrogate's posterior. However, the cost of GP inference grows cubically with the number of data points, which limits its applicability in real-time or embedded systems (e.g., spacecraft, edge devices).

Efficiency improvements have been pursued via sparse approximations<sup>51,52</sup>, discretization<sup>42,53,54</sup>, and simulation-based augmentation<sup>55–57</sup>. While effective, these methods either constrain representational flexibility or require significant additional resources (e.g., high-fidelity simulators).

Structured and mixed-domain optimization problems remain especially challenging. Existing approaches often rely on engineered kernels<sup>58,59</sup> or learned latent space projections<sup>60–64</sup>. However, these methods require training generative models, introduce interpretability challenges, and can be expensive to optimize. While alternatives to GPs exist, the GP formulation remains attractive because of its ease of implementation, convenient theoretical properties, and connection to exploratory behaviors in humans<sup>7,65</sup>.

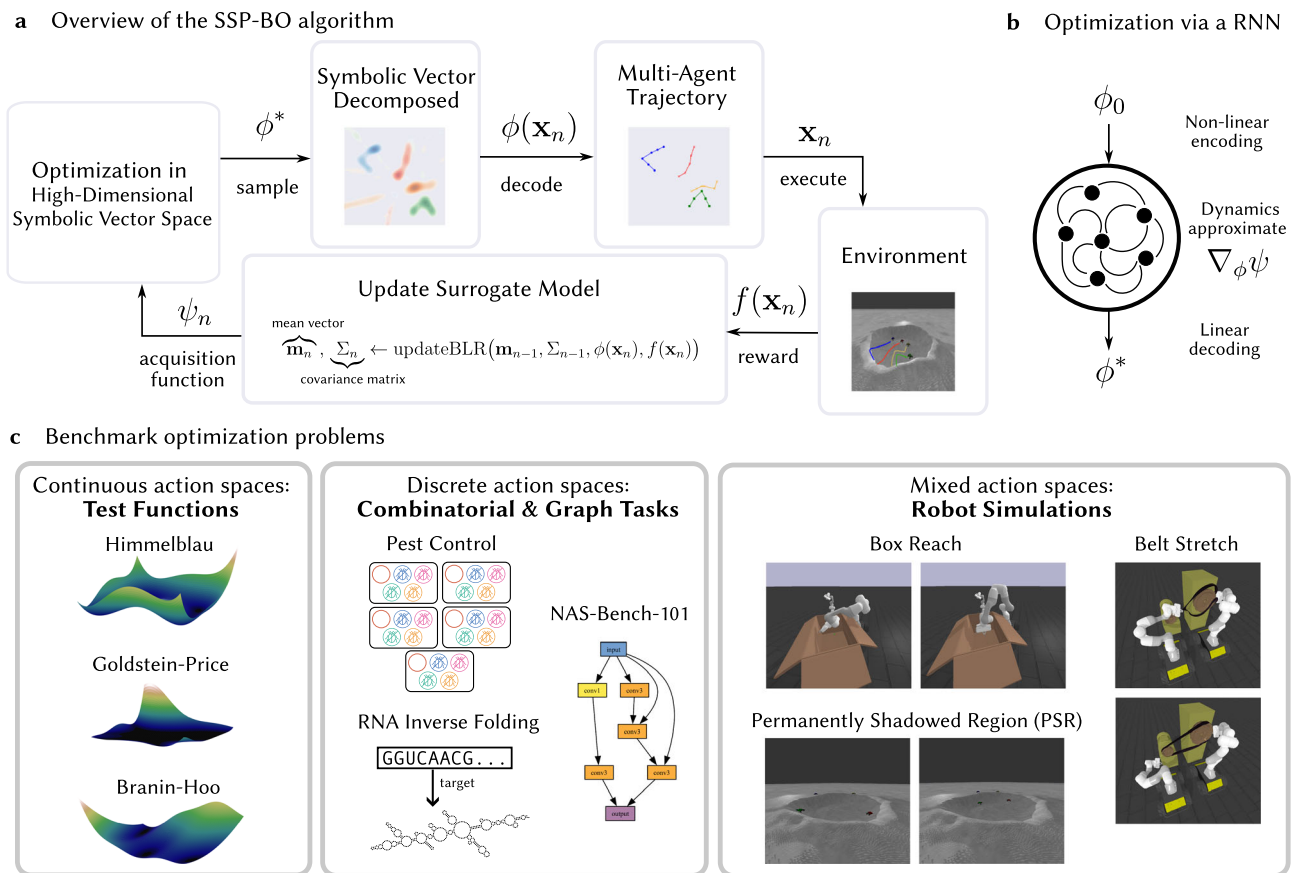
Positive-definite kernels are closed under addition and multiplication [66, section 6.2]. These algebraic operations form the basis for compositional kernels<sup>67,68</sup>, which can be used to encode inductive biases over structured data. This compositionality has been shown to improve generalization in machine learning domains<sup>59</sup>. Furthermore, in cognitive modeling, structured inductive biases reduce the amount of data required for learning<sup>5,69,70</sup>.

In kernel approximation methods, such as RFFs, compositional constructions are generally obtained using concatenation and tensor products of features [e.g.,<sup>10,71–73</sup>]. However, these operations result in feature dimensionality that grows linearly or multiplicatively with composition depth, leading to unbounded resource use as kernel complexity increases. A notable exception is the method proposed by ref. 74, which constructs product kernels using element-wise multiplication of complex-valued vectors—a strategy closely related to the FHRR binding operator.

In contrast, VSA-based embeddings used in SSP-BO maintain constant dimensionality through fixed-width algebraic operations, such as bundling and binding (a compressed tensor product). This enables scalable kernel approximation without sacrificing expressivity and supports compositional representations over arbitrary data types. Our method falls within the class of syntactic-driven kernels<sup>58,75</sup>, in which each structured representation induces a corresponding kernel function. Overall, VSAs preserve compositional expressiveness while keeping memory and compute costs bounded—an essential property for neuromorphic implementation.

This paper introduces SSP-BO, a BO algorithm that uses compositional kernel embedding based on VSA-derived representations. In sum, the main contributions of this work are:

- **Using hybrid cognitive representations of continuous trajectories and symbolic entities for BO.** VSAs allows us to address the long-standing problem of efficient BO in combinatorial, continuous, and mixed discrete-continuous spaces by leveraging VSAs with the same algorithm.
- **Employing neuroscience-inspired representations for BO.** We use Hexagonal-Spatial Semantic Pointers (Hex-SSPs), a VSA method for encoding continuous data, used in modeling activity of spatially sensitive neurons found in the hippocampus<sup>36</sup>.
- **Demonstrating constant time and memory sample selection complexity for BO.** SSP-BO reduces the complexity of selecting the next sample during BO from  $\mathcal{O}(n^3)$  and  $\mathcal{O}(n^2)$  in time and memory respectively, to constant in the number of samples



**Fig. 1 | Overview of the SSP-BO method and experiments.** **a** Our method conducts optimization in the space of vector symbolic representations. Upon obtaining an candidate vector, we decode it using vector symbolic operators. With that, we obtain an action vector  $\mathbf{x}_n$ , which can be executed in the environment. In this example, an action vector represents desired trajectories of robots searching for water in a crater. We obtain a reward  $f(\mathbf{x}_n)$  from the environment and incorporate it into the posterior distribution over the reward function. We update the posterior using Bayesian linear regression. We then optimize an information-theoretic acquisition function in the space of vector symbolic representations, produce the next embedded candidate vector, and so on. This way of conducting optimization yields a method with native support for compositionality and constant time and space complexity. **b** We demonstrate that a recurrent neural population can be used to optimize the objective in the embedding space. Neurons are connected all-

to-all, with connection weights set to approximate gradient ascent on the acquisition function. The selected sample  $\phi \in \mathbb{R}^d$  is linearly decoded from the population's activity. **c** The benchmark optimization problems that SSP-BO is tested on. On the left are three standard test functions for global optimization: the Himmelblau, Goldstein-Price, and Branin-Hoo functions. The SSP-BO method was tested on these functions with SSPs used to represent the 2D function input. In the middle are the discrete action space tasks: Pest Control, RNA Inverse Folding and NAS-Bench-101. On the right are the robotics environments: Box Reach, Belt Stretch, and Permanently Shadowed Region (PSR). In Box Reach, the objective is to quickly reach a target within a box. In Belt Stretch, two arms must collaborate to place an elastic belt on a motor and gear assembly. In PSR, multiple robots explore the permanently shadowed of a crater to find water.

collected and  $O(d^2)$  and  $O(d^2)$  in the embedding size, which remains fixed during optimization. We empirically support the theoretical claims that SSP-BO's run time is approximately constant in the number of collected samples, across optimization problems. We show this results in a total sample selection time reduction by a factor of  $3.1 - 205.5 \times$  (depending on the total number of samples and the problem size) compared to standard BO methods.

- **Validating the SSP-BO algorithm on neuromorphic computing hardware.** We provide a neural implementation of the sample selection in SSP-BO and demonstrate performance on two neuromorphic computing platforms (SpiNNaker and Loihi 2). Notably, this showcases the first implementation of BO on neuromorphic computing platforms.

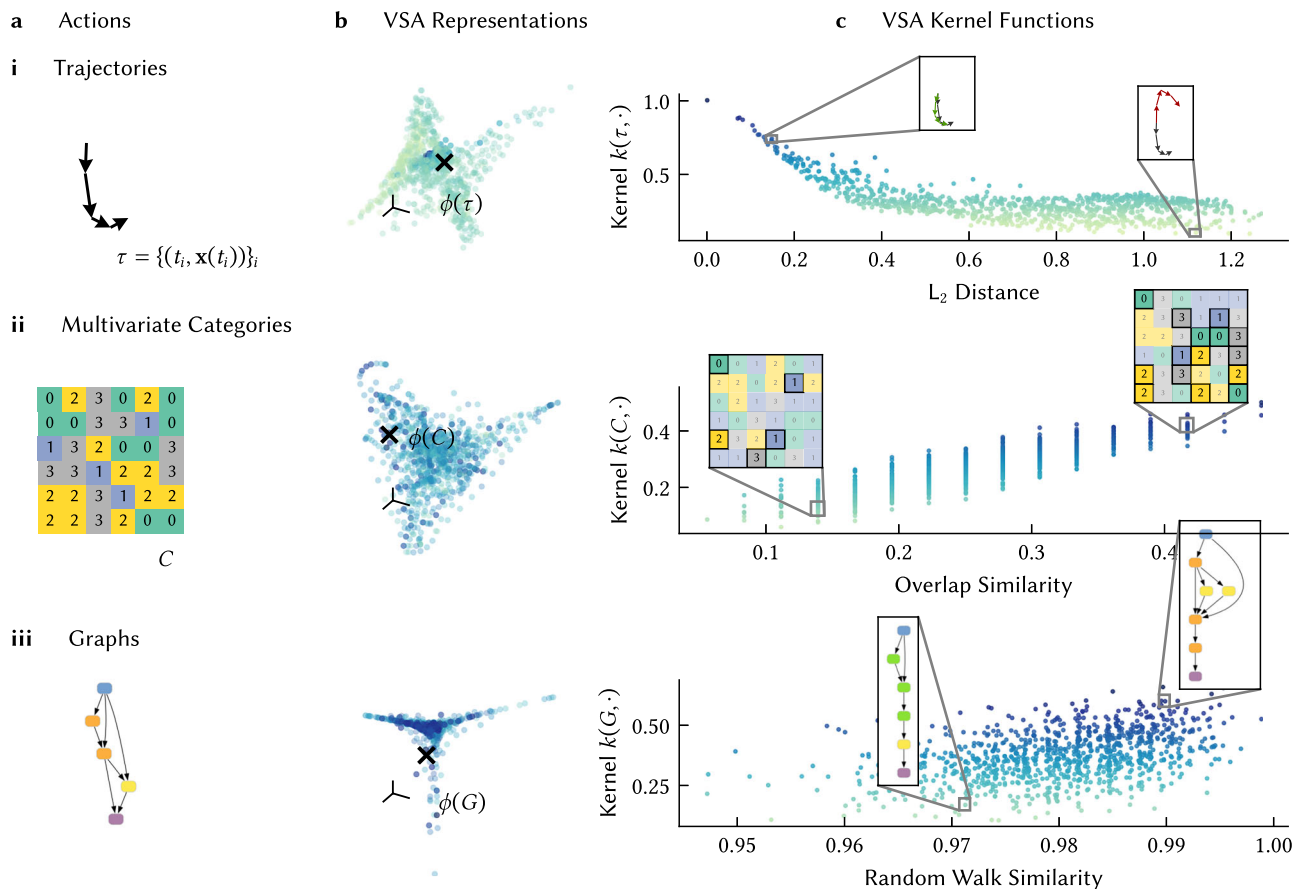
In requiring only fixed rather than growing resources, our approach enables deploying sophisticated methods for lifelong learning and autonomous exploration on resource-constrained platforms, including neuromorphic hardware. Beyond the specific results reported here, we hope this work illustrates a broader point: that the

architectural constraints of neuromorphic and biological systems (fixed resources, compositional structure, constant-time operations) need not limit the sophistication of the algorithms they support, and that established machine learning methods can benefit from the representational frameworks developed in these communities.

## Results

Our algorithm follows a Reinforcement Learning (RL) paradigm: an agent interacts with a (potentially stochastic) environment and seeks the most rewarding actions. The agent can observe the reward function by sampling actions  $\mathbf{x}$  from a domain,  $\mathcal{X}$ , up to some maximum sampling budget. After receiving an observation, the agent updates its surrogate model of the reward function. There is a need to balance exploration, i.e., searching for more rewarding actions, and exploitation, i.e., using the best rewarded actions seen so far. Actions are selected by optimizing an acquisition function that balances these factors (Equation (18)).

A key aspect of our approach is the use of a Vector Symbolic Algebra (VSA). Actions are mapped to a high-dimensional embedding space,  $\phi: \mathcal{X} \rightarrow \mathbb{R}^d$ , using a VSA. This mapping can be derived for



**Fig. 2 | VSA encodings and kernels for different action spaces.** **a** Example action spaces used in our optimization tasks, including (i) trajectory, (ii) combinatorial, and (iii) graph-based spaces. A reference action from each space is shown: (i) a trajectory  $\tau$  represented as waypoints connected by arrows, (ii) a combinatorial variable  $C$  displayed as a color-coded grid where each square represents a categorical value, and (iii) a directed graph representing a convolutional neural network architecture, with nodes as layers (colored by operation type) and arrows as connections<sup>103</sup>. **b** VSA-based representations of randomly sampled actions projected into a 3D space using locally linear embedding for visualization (actual representations are higher-dimensional). The black cross marks the reference action (e.g.,  $\tau$ ) and colors of other markers indicate the associated kernel value (e.g.,

$k(\tau, \cdot)$ ). **c** The kernel values under VSA encodings. Each point represents a randomly sampled action, with insets showing example actions (in (i) the reference trajectory is plotted underneath in black and in (ii) overlap with the reference variable is shown). The y-axis corresponds to the cosine similarity between the VSA representation of the reference action (from **a**) and other actions, while the x-axis corresponds to a standard distance or similarity measure specific to each action space: (i)  $L_2$  distance between trajectories, (ii) categorical overlap similarity, and (iii) a random walk similarity measure for graphs. These results show that VSA-based embeddings induce meaningful similarity measures aligned with known metrics in each space.

various structured action domains, as explained in the next section. The BO loop using VSA embeddings is presented in section “SSP-BO algorithm” and an overview of SSP-BO is given in Fig. 1. Empirical results begin in section “Experiments”.

### Vector symbolic algebra (VSA)

We introduce VSAs here, but readers interested in greater detail, including how to map them to spiking or non-spiking neural networks, can refer to other resources<sup>12,20,76–80</sup>. Below we provide the mathematical introduction to VSAs, intuition about the kernels induced by these structures are given in section “Methods”.

VSAs are algebras for capturing symbolic reasoning in high-dimensional vector spaces. In VSAs, symbols, represented as high-dimensional vectors (Fig. 2), can be manipulated by operations on those vectors. Cognitive models are constructed by combining vectors and operators, which can then be used to process stimuli. VSAs are typically equipped with four key operators: *bundling* ( $+$ ), *similarity* ( $\cdot$ ), *binding* ( $\otimes$ ), and *unbinding* ( $\oslash$ ). Some VSAs also define a *hiding* operator, which can be implemented using permutation matrices. HRRs, however, do not include a distinct hiding operator, as that can be achieved by binding. In this paper, we work with Holographic

Reduced Representations (HRRs), a type of VSA developed by ref. 12. In this framework, *bundling* is vector addition, *similarity* is cosine similarity, *binding* is circular convolution, and *unbinding* is circular correlation, which can be approximated by circular convolution with the pseudo inverse of a vector. Given a convolutional binding operator, it is common to improve the efficiency of the binding and unbinding operations by conducting them in the Fourier domain. Indeed, Fourier HRRs are a variation on HRRs that is conducted entirely in the Fourier domain, and has been used in a number of neuromorphic applications<sup>34,81,82</sup>.

It is sometimes necessary to represent indivisible concepts or objects that may serve as a referent to some high-level concept without defining its constituent elements. We could represent a concept or category, e.g., “retirement” or “animal”, by creating a discrete symbol that the system treats as having the semantic content of the concept it points to. Here discrete symbols are represented by vectors on the unit hyper-sphere. These are generated randomly by selecting their frequency components in the Fourier domain, and then converting them to the pseudo-time domain through the inverse Fourier transform. This differs from the original proposition of<sup>83</sup> for real-valued HRR symbols, who specified that

vectors should be drawn from the distribution,  $\mathcal{N}(0, d)$ . However, the representations we use for discrete symbols are equivalent to the inverse Fourier transform of Plate's "circular vectors" with the addition of conjugate symmetry.

Specifically, we generate a phase vector,  $\mathbf{a} \in \mathbb{R}^d$ , where elements of  $\mathbf{a}$  are sampled uniformly from the interval  $[-\pi, \pi]$ . Other sampling distributions can be used to generate  $\mathbf{a}$ , or it may be set some other way, e.g., end-to-end learning. We further constrain  $\mathbf{a}$  to have conjugate symmetry, to ensure the inverse Fourier transform does not generate imaginary components, and to be unit-length in the time domain, which places us in a strict subset of HRRs. We define our embedding of the symbol,  $s \in \mathcal{S}$ , in the high-dimensional space as:

$$\phi_{\mathcal{S}}(s) = \mathcal{F}^{-1}\{e^{i\mathbf{a}}\}, \quad (1)$$

where  $\mathcal{F}^{-1}$  indicates the inverse Fourier transform, and the exponential function is applied to each element of  $i\mathbf{a}$ . The kernel induced under this mapping is given in section "Kernels over Discrete Encodings".

To represent multiple entities, like symbols indicating the identities of several agents, we generate a unique  $\mathbf{a}_k$  for every entity. Thus, for a set of discrete symbols,  $\mathcal{S} = \{s_1, \dots, s_M\}$ , we generate a corresponding set of phase vectors,  $\{\mathbf{a}_1, \dots, \mathbf{a}_M\}$ . Discrete symbol vectors play multiple roles within VSAs. They can be used to represent the values of categorical data, but they can also be used to impose structure on a vector representation.

Chiefly, structure is imposed by using discrete symbol vectors to define the *slot* in *slot-filler* relationships. By analogy to programming languages, slots are like the names of variables, and the fillers are the values those variables take on. For example, sentences can be represented in a slot-filler manner with slots such as "subject" or "verb" specifying the functional roles of particular discrete data, such as "Bob" or "throw,"

$$\text{subject} \otimes \text{Bob} + \text{verb} \otimes \text{throw} + \text{object} \otimes \text{ball}. \quad (2)$$

More generally, consider a multi-category or combinatorial variable  $C$  (e.g., an object with multiple properties that take on discrete values). There is a set of category slots  $\phi_{\mathcal{S}}(s_i)$  (e.g.,  $s_1 = \text{color}$  and  $s_2 = \text{classification}$ ) each of which has an associated value  $\phi_{\mathcal{V}_i}(v_{i,j})$  (e.g., the possible colors are  $v_{1,1} = \text{red}$ ,  $v_{1,2} = \text{blue}$ , ... and possible classifications are  $v_{2,1} = \text{door}$ ,  $v_{2,2} = \text{box}$ , ... etc.). The mappings are labeled with  $\mathcal{S}$  and  $\mathcal{V}_i$  to indicate that there are different dictionaries for category slots and value types. The complete combinatorial variable  $C$  can be represented as

$$\phi_C(C) = \sum_{i=1}^{\# \text{ of categories}} \phi_{\mathcal{S}}(s_i) \otimes \phi_{\mathcal{V}_i}(v_{i,j}) + \kappa \bigotimes_{i=1}^{\# \text{ of categories}} \phi_{\mathcal{V}_i}(v_{i,j}). \quad (3)$$

The first term bundles all bound slot-filler pairs,  $\kappa = 0.1$  is a weighting constant, and the second term is the bound combination of all values.

As discussed later in section "Data Structures as Approximate Compositional Kernels", purely bundling variables restricts the representation to additive kernels. To represent multiplicative kernels and capture interactions between input dimensions, the all-bound term is required. This could be generalized to sum over  $n$ -gram bound groups, assuming the category slots are ordered in a meaningful way<sup>84,85</sup>. An example of multi-category embeddings is shown in Fig. 2(ii).

VSAs have been previously used to encode graphs<sup>78,86,87</sup>. In section "Experiments", we consider an optimization problem defined over

directed acyclic graphs with  $M$  labeled nodes, which we represent by

$$\phi_G(G) = \frac{1}{M} \sum_{i=1}^M \phi_{\mathcal{N}}(n_i) \otimes \left( \text{node\_label} \otimes \phi_{\mathcal{L}}(l_i) + \text{tail} \bigotimes \sum_{j \text{ where } i \rightarrow j} \phi_{\mathcal{N}}(n_j) \right) + \kappa \bigotimes_{i=1}^M \phi_{\mathcal{N}}(n_i) \otimes \phi_{\mathcal{L}}(l_i) \otimes \sum_{j \text{ where } i \rightarrow j} \phi_{\mathcal{N}}(n_j), \quad (4)$$

where  $\text{tail}$  and  $\text{node\_label}$  are slot vectors,  $n_i, n_j \in \mathcal{N}$  denote nodes, and  $l_i \in \mathcal{L}$  is a label associated with the vertex. Nodes disconnected from all others are removed from the graph. Note that, like the multi-category embeddings, there is an all-bound term weighted by a constant  $\kappa = 0.1$ .

Moving from discrete symbol representations to discrete ordered representations can be thought of as constructing vector representations of integers. An integer,  $j \in \mathcal{J} \subseteq \mathbb{Z}$ , can be represented by first selecting a basis vector  $\mathcal{F}^{-1}\{e^{i\mathbf{a}_j}\}$ , a randomly generated symbol as described in the previous section (i.e.,  $\mathbf{a}_j$  is a random phase vector). To represent an integer,  $j$ , the basis vector is bound with itself  $j$  times. We denote this as:

$$\phi_j(j) = \bigotimes^j \mathcal{F}^{-1}\{e^{i\mathbf{a}_j}\} = \mathcal{F}^{-1}\{e^{i\mathbf{a}_j j}\}. \quad (5)$$

In contrast to discrete symbols, integer representations are used to represent values that are ordered and have a meaningful distance (and hence similarity) metric.

To extend iterated binding to real-valued data we exploit the fact that convolution in the time domain is multiplication in the Fourier domain. Integer representations are trivially extended to real-valued data to produce a method known as *fractional binding*<sup>33,35</sup> or *fractional power encoding*<sup>29,34</sup>. Because some of these methods were developed in the context of the Semantic Pointer Architecture<sup>19</sup>, they are sometimes known as Spatial Semantic Pointers (SSPs)<sup>35</sup>. We can thus represent real-valued data  $x \in \mathcal{X} \subseteq \mathbb{R}$  as

$$\phi_{\mathcal{X}}(x) = \mathcal{F}^{-1}\{e^{i\mathbf{a}_{\mathcal{X}} x}\}, \quad (6)$$

where  $\ell \in \mathbb{R}^+$  is the length scale of the representation,  $\mathbf{a}_{\mathcal{X}}$  is the phase vector, and the exponentiation is applied element-wise, as above. This is similar to the method of RFFs<sup>9</sup>, in that it maps scalar inputs to a higher-dimensional feature space via complex exponentials of linear projections. However, unlike RFFs these representations are used with the HRR algebra to create embeddings of compositional structures.

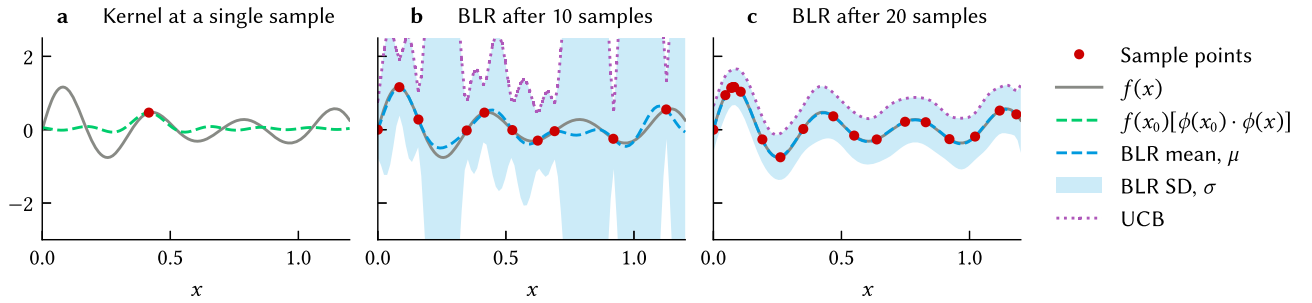
Provided that two variables are bound using the same basis vector, the dot product between fractional bound representations induces a kernel,  $k(\mathbf{x}, \mathbf{x}') = \phi_{\mathcal{X}}(\mathbf{x})^T \phi_{\mathcal{X}}(\mathbf{x}')$ . The expected shape of the kernel is a function of the sampling distribution for the phase vector components. Thus, the similarity between continuous variables can be tuned to the needs of a particular application, as typically done with RFFs. In the case of  $(a)_i$  - Uniform $[-\pi, \pi]$ , the induced kernel is the sinc function<sup>27</sup>. This is a useful kernel because it converges to a point mass in the small bandwidth limit, i.e.,  $\text{sinc}((i-j)/l) \rightarrow \delta(i-j)$  as  $l \rightarrow 0$ .

This encoding can be extended to  $m$  - dimensional data,  $\mathbf{x} \in \mathcal{X} \subseteq \mathbb{R}^m$ , by replacing the phase vector,  $\mathbf{a}_{\mathcal{X}}$ , with a phase matrix,  $\mathbf{A}_{\mathcal{X}} \in \mathbb{R}^{d \times m}$ ,

$$\phi_{\mathcal{X}}(\mathbf{x}) = \mathcal{F}^{-1}\{e^{i\mathbf{A}_{\mathcal{X}} \mathbf{x}}\}. \quad (7)$$

In this case, the phase matrix is selected such that the columns are independently generated random vectors. We refer to such an encoding scheme as Random SSPs (Rand-SSPs).

**Hexagonal SSPs.** In this work, we use a phase matrix that has been structured in a particular way. Specifically, the phase matrix is set so that the induced kernel is a sum of periodic kernels that mimic the neural activity of grid cells in the medial entorhinal cortex<sup>36</sup>. We have chosen this structure because it allows SSPs to be efficiently represented by neural populations and, as features for deep networks,



**Fig. 3 | Function posterior estimated using BLR and SSP encoding as the number of samples increases.** Red dots indicate sample points, the blue line indicates the estimated function mean, the shaded regions indicate the prediction standard deviation, and the gray line is the true function. **a** The estimated function

using only one observation, where uncertainty is undefined. **b** After ten samples the mean begins to approach the true function, although the uncertainty is high. **c** With 20 samples, we obtain an accurate function estimate with reasonable uncertainty bounds.

outperforms Rand-SSPs on a number of classification and regression tasks<sup>36,88,89</sup>. We call the resulting feature representation Hexagonal SSPs (Hex-SSPs). Section “Hexagonal SSPs” provides the details of the Hex-SSP construction and a derivation of the kernel induced under this embedding.

If we consider the space of trajectories,  $\mathcal{T}$ , we can represent a trajectory  $\tau = [\mathbf{x}(t)]_{t \in [0, T]}$  with SSPs as a bundle of time and location representations bound together in a single vector<sup>90</sup>:

$$\phi_{\mathcal{T}}(\tau) = \int_0^T \phi_{\mathcal{X}}(\mathbf{x}(t)) \otimes \phi_{\mathcal{T}}(t) dt, \quad (8)$$

where  $t$  is time and  $T$  is the maximum trajectory duration. Pragmatically, instead of computing the exact integral over the trajectory  $\tau$ , we often sample the trajectory and encode a bundle of waypoints bound with time:

$$\phi_{\mathcal{T}}(\tau) = \sum_{i=1}^n \phi_{\mathcal{X}}(\mathbf{x}(t_i)) \otimes \phi_{\mathcal{T}}(t_i). \quad (9)$$

If we want to represent trajectories of multiple agents, we can generate random symbol vectors for each agent, bind them to the corresponding trajectory embeddings, and represent the collection in one bundled vector.

$$\sum_{i=1}^{\text{\# of agents}} \phi_{\text{agts}}(s_i) \otimes \phi_{\mathcal{T}}(\tau_i), \quad (10)$$

where  $\phi_{\text{agts}}(s_i)$  are randomly generated symbol-vectors for the agents whose trajectories are being optimized.

The encoding in Equation (10) serves as the projection into the feature space used in the experiments presented in this work. Note that regardless of the trajectory length or the number of agents involved, the dimensionality of the SSP vector representation does not change.

### SSP-BO algorithm

**Algorithm 1.** SSP Bayesian Optimization (SSP-BO).  $N$  is the total sampling budget for the algorithm,  $\beta$  is the inverse variance parameter,  $\alpha$  was chosen to be 0.01,  $f(\cdot)$  is the target function being optimized, and  $\mathcal{X}$  is the domain of  $f(\cdot)$ .  $\gamma_c$  is a scaling parameter that can be used to further control the influence of the exploration term.

1: **Procedure** SSP-BO ( $N, f(\cdot), \frac{1}{\beta}, \alpha, \lambda$ )

2:  $\mathbf{m}_0, \Sigma_0, \Sigma_0^{-1} \leftarrow \mathbf{0}, \alpha \mathbf{I}, \alpha^{-1} \mathbf{I}$

3: **for**  $k \leftarrow 1$  to  $N$  **do**

4:  $\mu_n(\theta) \triangleq \mathbf{m}_n \cdot \theta$

5:  $\sigma_n(\theta) \triangleq \sqrt{\frac{1}{\beta} + \theta^T \Sigma_{n-1}^{-1} \theta}$

6:  $\theta^* \leftarrow \arg \max_{\theta \in \mathbb{R}^d} [\mu_n(\theta) + \lambda_n \sigma_n(\theta)]$   
 7:  $\mathbf{x}_n \leftarrow \arg \max_{\mathbf{x} \in \mathcal{X}} \phi(\mathbf{x}) \cdot \theta^* \quad \triangleright$  Decode optimal SSP  
 8:  $f_n \leftarrow f(\mathbf{x}_n)$   
 9:  $\mathbf{m}_n, \Sigma_n, \Sigma_n^{-1} \leftarrow \text{updateBLR}(\mathbf{m}_{n-1}, \Sigma_{n-1}, \Sigma_{n-1}^{-1}, \mathbf{x}_n, f_n)$   
 $\triangleright$  See eqs. (14)–(16)

10: **end for**

11: **end procedure**

SSP-BO follows the standard BO framework, adapted to operate over VSA representations (section “Kernel Approximations with VSAs”). At each iteration, the agent selects an action  $\mathbf{x}_n \in \mathcal{X}$  by optimizing an acquisition function, observes a reward  $f(\mathbf{x}_n)$ , and updates its surrogate model. This process repeats until a sampling budget  $N$  is exhausted.

**Surrogate model.** We model the reward function using Bayesian linear regression (BLR; Fig. 3), which provides a probabilistic surrogate over VSA embeddings. Each action  $\mathbf{x}$  is mapped to a fixed-dimensional vector  $\phi(\mathbf{x}) \in \mathbb{R}^d$ , and the predicted reward is modeled as a Gaussian distribution:

$$\hat{f}(\mathbf{x}) \sim \mathcal{N}(\mu(\mathbf{x}), \sigma^2(\mathbf{x})). \quad (11)$$

The parameters of this distribution are computed using standard BLR formulas [66 Section 3.3]:

$$\mu_n(\mathbf{x}) = \mathbf{m}_{n-1}^T \phi(\mathbf{x}), \quad (12)$$

$$\sigma_n^2(\mathbf{x}) = \frac{1}{\beta} + \phi(\mathbf{x})^T \Sigma_{n-1} \phi(\mathbf{x}), \quad (13)$$

where  $\mu_n(\mathbf{x})$  is the posterior mean of the predicted reward at  $\mathbf{x}$ , parameterized by the mean vector  $\mathbf{m}_{n-1}$ , and  $\sigma_n^2(\mathbf{x})$  is the posterior variance, parameterized by the covariance matrix  $\Sigma_{n-1}$ . The precision parameter  $\beta$  is set as the inverse variance of the observed rewards and is estimated from the initial samples. The mean vector and covariance matrix are updated incrementally with each new observation.

**Online updates.** We use rank-one updates to update the inverse covariance matrix  $\Sigma_n^{-1}$  and thus update the covariance matrix  $\Sigma_n$  online via the matrix inversion lemma (Equation (15)), avoiding costly inversions. Online updates to the BLR parameters are given by

$$\Sigma_n^{-1} = \Sigma_{n-1}^{-1} + \beta \phi(\mathbf{x}_n) \phi(\mathbf{x}_n)^T, \quad (14)$$

$$\Sigma_n = \Sigma_{n-1} - \frac{\Sigma_{n-1} \phi(\mathbf{x}_n) \phi(\mathbf{x}_n)^T \Sigma_{n-1}}{\frac{1}{\beta} + \phi(\mathbf{x}_n)^T \Sigma_{n-1} \phi(\mathbf{x}_n)}, \quad (15)$$

$$\mathbf{m}_n = \Sigma_n \Sigma_{n-1}^{-1} \mathbf{m}_{n-1} + \Sigma_n \beta \phi(\mathbf{x}_n) f(\mathbf{x}_n). \quad (16)$$

**Acquisition function.** Actions are selected by maximizing an acquisition function,  $\psi_n(\mathbf{x})$ . We use the Upper Confidence Bound [UCB; <sup>91–94</sup>] criterion to balance exploration and exploitation. The acquisition function at iteration  $n$  is:

$$\psi_n(\mathbf{x}) = \mu_n(\mathbf{x}) + \lambda \sigma_n(\mathbf{x}) \quad (17)$$

$$= \mathbf{m}_{n-1}^T \phi(\mathbf{x}) + \lambda \sqrt{\frac{1}{\beta} + \phi(\mathbf{x})^T \Sigma_{n-1} \phi(\mathbf{x})}. \quad (18)$$

where  $\lambda = 1$  is a scalar that governs the trade-off between uncertainty-driven exploration and predicted reward.

**Optimization in embedding space.** To select the next action, we optimize the acquisition function in the high-dimensional VSA space:

$$\text{Optimize : } \theta^* = \arg \max_{\theta \in S^d} \mathbf{m}_{n-1}^T \theta + \lambda \sqrt{\beta^{-1} + \theta^T \Sigma_{n-1} \theta}, \quad (19)$$

$$\text{Decode : } \mathbf{x}_n = \arg \max_{\mathbf{x} \in \mathcal{X}} \theta^* \cdot \phi(\mathbf{x}). \quad (20)$$

where  $S^d$  is the  $d$ -dimensional unit hypersphere. Note that  $\theta^*$  may not correspond exactly to any valid action embedding – i.e.,  $\theta^*$  might not lie in the set  $\{\phi(\mathbf{x}) : \mathbf{x} \in \mathcal{X}\}$ . Thus, we decode it by selecting the action whose embedding is most similar in feature space. This decoding step is conceptually similar to projection methods in constrained optimization, but in our case is based on similarity rather than geometric projection.

By operating in the embedding space, SSP-BO transforms a complex optimization problem in a potentially structured or discrete action space,  $\mathcal{X}$ , into a continuous optimization problem on  $S^d$ , at the cost of approximation. The objective is smooth and defined on a compact domain, lending itself to efficient optimization via gradient ascent or neural dynamics. This formulation also lends itself to neuromorphic implementation, as the same recurrent neural architecture can perform sample selection across diverse data types and structures (e.g., discrete, continuous, sets, graphs, and trajectories). However, the task of decoding  $\theta^*$  into an appropriate action,  $\mathbf{x}^*$ , can be challenging, see section “Feature Decoding” for more details.

The complete algorithm is given in Algorithm 1.

## Experiments

We now demonstrate that VSA-based representations, when used in the acquisition function Equation (18), support effective optimization. We compare SSP-BO against various BO baselines that optimize directly in the action space  $\mathcal{X}$ .

Our experiments span a range of action types and problem structures (Fig. 1(c)). First, we evaluate SSP-BO on continuous function optimization using the Hex-SSP representation. Second, we test SSP-BO on discrete action spaces, including combinatorial optimization and a neural architecture search problem. These tasks assess whether VSA representations support efficient optimization over symbolic and structured domains. Third, we evaluate SSP-BO on multi-agent trajectories, demonstrating the method’s ability to optimize in mixed, high-dimensional spaces.

In all domains, we find that SSP-BO performs better than or comparably to BO baselines. We also conduct runtime profiling and complexity analysis, showing that SSP-BO yields significant computational advantages due to its fixed-size embedding and efficient updates. Finally, we demonstrate that the algorithm can be implemented on neuromorphic hardware using spiking neurons, highlighting its potential for energy-efficient, brain-inspired optimization.

Previously, randomly generated SSPs have been used as a feature space for BO<sup>95</sup>. In this work, we show that Hex-SSP features are more effective than Rand-SSP features for BO, although both can be used. Because we test these algorithms on standard target functions, we know what the maximum value of these functions is, so we can score the algorithms on the basis of regret. Regret is the difference between the reward obtained,  $f(x_n)$ , and the best reward,  $f(x^*)$ . We report the average regret at sample number  $n$ ,  $R_n = \frac{1}{n} \sum_{i=1}^n f(x^*) - f(x_i)$ . Lower values of this quantity are better.

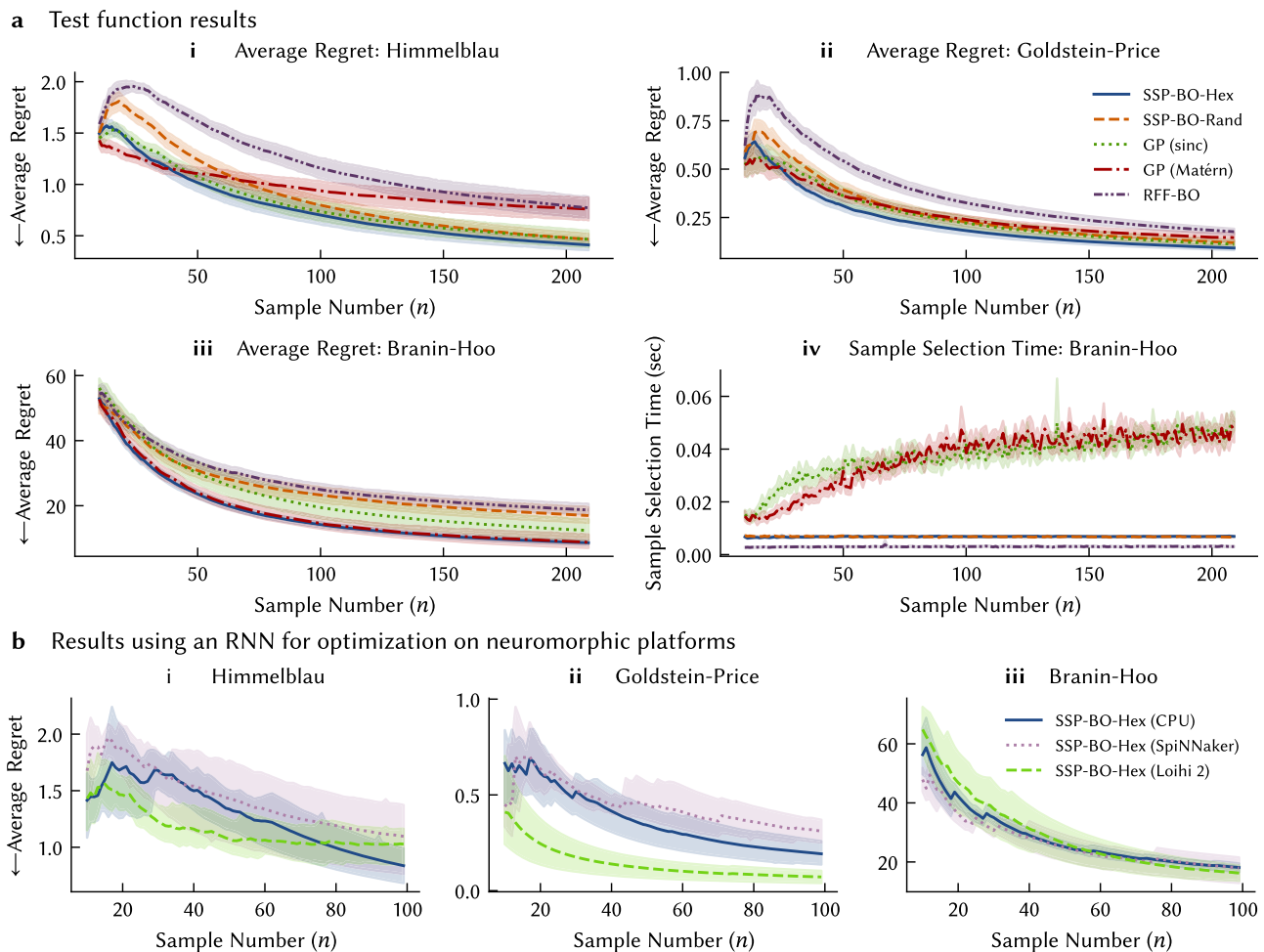
Figure 4a shows the performance of SSP-BO on three common global optimization functions, as well as the time complexity of the algorithms compared to a baseline algorithm implemented using GP and a GP approximated with RFFs (see section “Baseline Implementations for BO” and Supplementary Note 3, “Baseline Methods” for details). As anticipated, SSP-BO has constant time complexity, and constant memory complexity (see Supplementary Fig. 1). Note that the time for RFF-BO is also constant, as it too uses a kernel approximation. It is in fact slightly faster than SSP-BO because RFF-BO optimizes in the original action space and therefore does not require a decoding step. However, RFF-BO has higher regret on all test functions. SSP-BO with Hex-SSPs achieves the best regret performance. Furthermore, we find that the Hex-SSP results were stable even when reduced to 25 dimensions (Supplementary Fig. 3).

We evaluate our approach on two combinatorial optimization tasks: a synthetic pest control task<sup>96</sup> and an RNA inverse folding task<sup>97</sup>. For comparison, we use benchmark results from <sup>98</sup>, which include various GP-based Bayesian optimization methods such as COMBO<sup>96</sup>, BOILS<sup>99</sup>, BODI<sup>100</sup>, and BOSS<sup>101</sup>. Each method employs different combinatorial kernels and optimization algorithms that operate directly in the combinatorial action space (see Supplementary Table II). Some methods use computational strategies to manage the cubic scaling of exact GP inference (e.g., Casmopolitan<sup>102</sup> dynamically constructs the GP within local trust regions). In contrast, our method approximates a GP via VSA encodings and performs gradient-based optimization directly in the embedding space.

In Fig. 5a, we report the best objective value rather than regret, as the true optimal value is unknown for these tasks. In these tasks, lower objective values are better. Within a budget of  $N = 200$ , our approach achieves a best objective value within the 95% confidence intervals of the top-performing BO methods in ref. <sup>98</sup>, demonstrating competitive performance while substantially reducing computation time. The total time spent by SSP-BO planning actions and updating its surrogate model is 205 to 3891 times faster than the total time for the other BO methods on the pest control task and 176 to 2763 times faster on the RNA inverse folding task. While SSP-BO’s sample efficiency is worse than some methods on the pest control task, it still achieves a good objective value in a fraction of the time—even when function evaluation time is included. In particular, the total time SSP-BO takes to sample and evaluate 200 samples is at least 30 times less than the time the best-performing baseline (SSK) takes to sample and evaluate 100 samples.

The time reduction achieved by SSP-BO stems from three key factors: (1) efficient feature similarity computation compared to expensive kernels (e.g., SSK), (2) fast optimization in a smooth continuous space rather than slow genetic algorithms or hill climbing in discrete combinatorial spaces, and (3) avoiding the cubic scaling of exact GP inference through VSA kernel approximations.

To further validate our method, we apply it to a graph optimization task, NAS-Bench-101<sup>103</sup>, a benchmark for Neural Architecture Search (NAS). The goal is to find the network architecture that achieves highest accuracy on CIFAR-10. This requires optimizing over a structured, discrete search space where architectures are represented as directed acyclic graphs with varying layer types. Examples of graphs in the action space are given in Fig. 2.



**Fig. 4 | SSP-BO offers bounded execution time and sample selection on neuromorphic hardware, while achieving regret comparable to standard implementations.** **a** Plots (i–iii) show the average regret  $R_t$  (lower is better); plot (iv) shows the sample selection time (lines 4–7 and 9 in Algorithm 1), which includes the decoding step and does not include the function evaluation time. Shaded regions are 95% bootstrap confidence intervals for 30 trials with sampling budgets of  $N = 200$ . There are an additional 10 random samples that are used to initialize the

surrogate models. This stage is not included in the plots, but the initial samples do contribute to the computed regret. SSP-BO-Hex achieves better or equal regret performance to GP (Matérn) while showing a substantial improvement in selection time. **b** Results when sample selection is performed by a spiking RNN. Plots (i–iii) show the average regret and 95% bootstrap confidence intervals for 5 trials with sampling budgets of  $N = 100$  using different hardware to simulate the RNN.

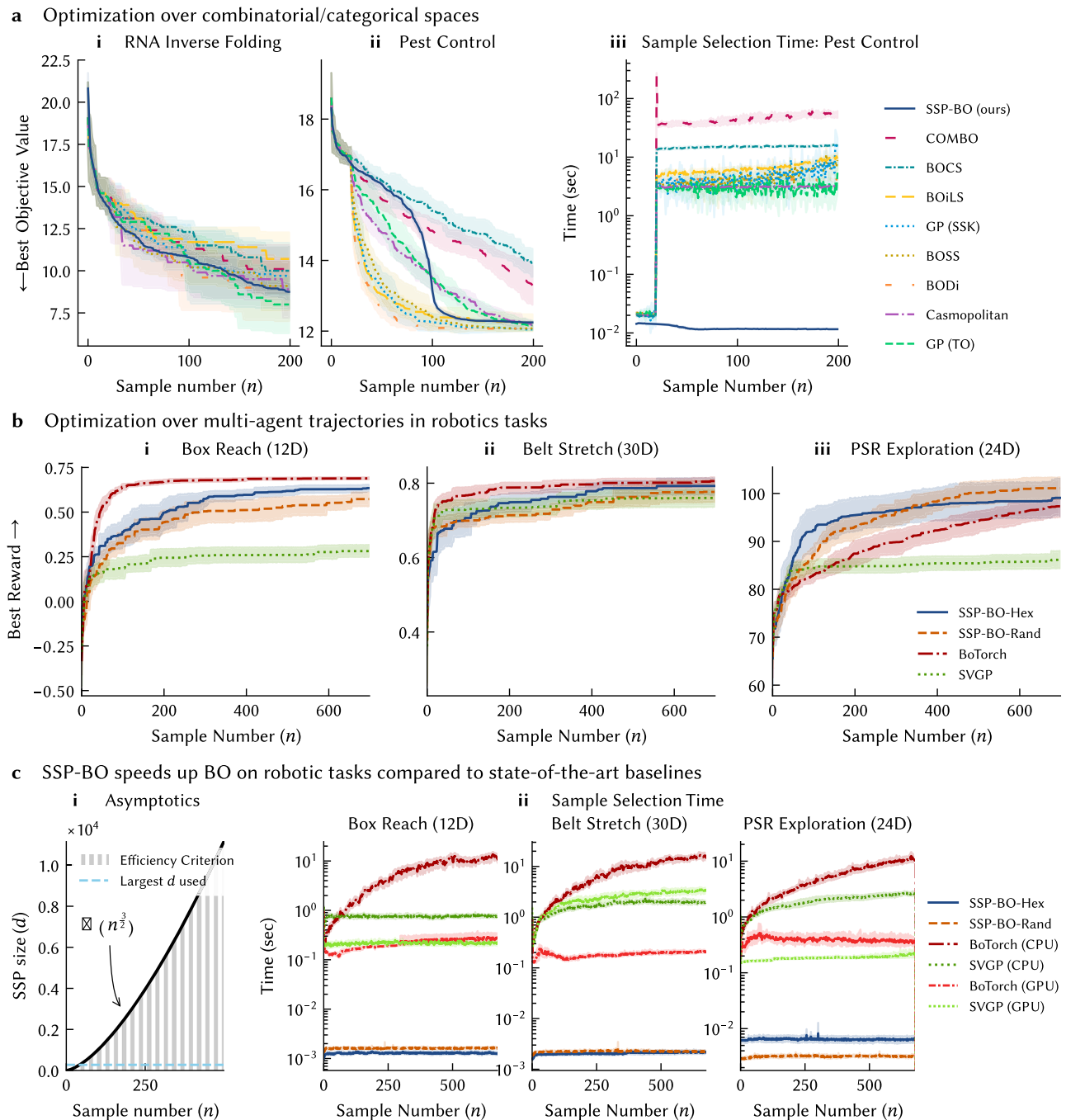
During optimization, we query the validation accuracy of selected architectures and report here the best validation and test accuracies found, as is standard. The results are given in Table 1. The accuracies obtained by SSP-BO are within the standard deviation of several existing BO NAS methods (e.g., NASBOT<sup>104</sup>, NAO<sup>62</sup>, BANANAS<sup>105</sup>, and BONAS<sup>106</sup>). While our approach performs worse than state-of-the-art NAS methods such as AG-Net<sup>107</sup>, a generative approach to NAS, and DiNAS<sup>108</sup>, a graph diffusion algorithm, it is competitive while maintaining efficiency and suitable to neuromorphic implementation. Most importantly, these results demonstrate that our method is capable of optimization over structured spaces like neural architecture graphs.

Having demonstrated that Hex-SSPs are a viable and efficient feature representation for BO and that VSAs can support BO over structured discrete spaces, we now move onto optimization in structured continuous spaces using hybrid continuous-symbolic representations. Specifically, we seek to optimize the trajectories of multiple robots, with the encoding in Equation (10), in three tasks relevant to robotics and machine learning. The first task, Box Reach, is a pick-and-place task. Two manipulators must learn to reach into a vessel. Pick-and-place tasks have multiple applications, from warehouse automation to laboratory sample handling. In the next scenario, Belt Stretch, two mobile manipulators are tasked with replacing a belt on a belt-

drive mechanism. Manipulating non-rigid objects is a challenging task and has applications in household maintenance, manufacturing and repair of machinery, and elder care, amongst others. The third task involves planning trajectories through the Permanently Shadowed Region (PSR) of a crater that maximize the observed counts from a simulated Neutron Spectrometer. Lunar prospecting is just one example of a coordinated, multi-agent, active learning task, but applications generalize to other information-gathering tasks such as resource mapping, search and rescue, and system identification.

We compare SSP-BO against a state-of-the-art baseline, BoTorch, and BO using Sparse Variational Gaussian Processes (SVGP). SVGP has constant time and memory complexity. The hyperparameters of SVGP are set to match the memory usage of SSP-BO.

In these test cases, we do not know what the highest possible reward is and so we report the best scoring reward achieved by sample number  $n$  rather than regret. In these tasks, higher rewards are better. Figure 5b shows the evolution of the best discovered reward by SSP-BO and the baseline algorithms for 10 trials. In two cases we are achieving statistically equivalent performance to the baseline BO algorithm and in all scenarios SSP-BO has equivalent or superior terminal reward to SVGP. However, SSP-BO is substantially faster than BoTorch and even SVGP (see Fig. 5c).



**Fig. 5 | VSA allow representation and fast optimization over combinatorial spaces.** **a** VSAs allow representation and optimization over combinatorial/categorical spaces. Plots (i–ii) show the best objective function value (lower is better for these two tasks); plot (iii) shows sample selection time on a log scale. In all figures, shaded regions are 95% bootstrap confidence intervals for 10 trials with sampling budgets of  $N = 200$ . SSP-BO again shows a substantial improvement in sample selection times. **b** SSP-BO comparable to state-of-the-art baselines on robotics tasks. (i–iii) show the performance of SSP-BO and baselines in terms of largest reward achieved as a function of the number of samples. In all plots, the lines show the mean of 10 runs (each run initialized with a different random seed); shaded regions indicate 95% confidence intervals. BoTorch leads on a simple Box Reach task (i), SSP-BO attains comparable performance on the more advanced Belt Stretch task with deformables (ii), and finally SPP-BO outperforms BoTorch and

other baselines on the PSR planetary exploration task with four agents (iii). **c** SSP-BO speeds up BO on robotic tasks. (i) The values of SSP dimensionality,  $d$ , and number of samples,  $n$ , for which SSP-BO is more efficient than exact GP-based BO in terms of time complexity asymptotics (shown by the shaded region). Exact GP-BO has an  $\mathcal{O}(n^3)$  time complexity, while SSP-BO is constant in  $n$  but quadratic in  $d$ , i.e.,  $\mathcal{O}(d^2)$ . Consequently, the instantaneous sample point selection for SSP-BO is an improvement over naive GP BO when  $d < n^{\frac{3}{2}}$  (the efficiency criterion). The dashed blue line shows the maximum SSP dimensionality used in our experiments. For this value, the theoretical break-even point is  $\approx 70$  samples. (ii) The time to compute each next sample for SSP-BO, BoTorch, and SVGP for the Box Reach, Belt Stretch, and PSR Exploration tasks on a logarithmic scale. Lines show the mean of 10 runs; shaded regions indicate 95% confidence intervals. In these experiments, SSP-BO offers significant reduction in compute time.

**Table 1 | Reported mean validation and testing accuracies, along with standard derivations, obtained on NAS-Bench-101 over 10 trials for different algorithms**

| Algorithm              | Val. Acc (%)        | Test Acc (%)        | # Samples |
|------------------------|---------------------|---------------------|-----------|
| <b>Optimum</b>         | <b>95.06</b>        | <b>94.32</b>        |           |
| NASBOT <sup>104</sup>  | –                   | 93.73               | 150       |
| NAO <sup>62</sup>      | 94.66 ± 0.14        | 93.49 ± 0.59        | 192       |
| BANANAS <sup>105</sup> | 94.73 ± 0.17        | 94.09 ± 0.19        | 192       |
| BONAS <sup>106</sup>   | –                   | 94.07 ± 0.24        | 200       |
| AG-Net <sup>107</sup>  | <b>94.90 ± 0.22</b> | <b>94.18 ± 0.10</b> | 192       |
| DiNAS <sup>108</sup>   | <b>94.98 ± 0.17</b> | <b>94.27 ± 0.20</b> | 150       |
| SSP-BO (ours)          | 94.44 ± 0.20        | 93.88 ± 0.21        | 200       |

NASBOT results are averaged over 200 trials and BONAS results are averaged over 50 trials instead of 10, per the cited papers.

**Table 2 | The energy use per optimization step of SSP-BO when deployed on neuromorphic hardware**

| Hardware  | Number of neurons | Energy (J/sample)                  |
|-----------|-------------------|------------------------------------|
| CPU       | —                 | $(234 \pm 64) \times 10^{-3}$      |
| SpiNNaker | 385               | $(1.244 \pm 0.005) \times 10^{-3}$ |
| Loihi 2   | 440               | $(3.760 \pm 1.30) \times 10^{-3}$  |
| Loihi 2   | 880               | $(7.451 \pm 0.751) \times 10^{-3}$ |

The CPU version implements the gradient-based algorithm on standard hardware. All models use SSP size  $d = 55$ . The model run on Loihi2 was sufficiently small that the in-built power profiler was unable to register its power consumption. Power results are from running a larger model and then scaling down consumption accordingly.

Sample selection in SSP-BO is made possible on neuromorphic hardware by optimizing directly within the high-dimensional embedding space. This avoids operating in the complex, discontinuous action space and allows gradient-based optimization to be carried out through the dynamics of a recurrent spiking neural network. We implemented this approach on SpiNNaker and Loihi 2, demonstrating that efficient sample selection is achievable on both platforms. Implementation details are provided in section “Neuromorphic Implementation for SSP-BO”.

Results on the global test optimization functions are shown in Fig. 4(b), which illustrates regret reduction on both platforms. Note that the results for Loihi 2 are better than those for SpiNNaker. This is because we were able to run a larger model on Loihi 2 (880 neurons versus 385) and use a more expressive neuron model (see Supplementary Note 2, “Hardware”). Energy consumption per optimization step is summarized in Table 2. The improvement factor is 30.4–62.2 for Loihi 2 and 188.1 for SpiNNaker, demonstrating the substantial energy-efficiency gains of the neuromorphic implementations compared to the CPU baseline.

Figure 5c illustrates the growth of the wall-clock time for selecting the next sample location. For BoTorch, this grows with the number of samples, but for SSP-BO and SVGP this is effectively constant in all simulation scenarios (see also Fig. 4aiv). SSP-BO provides a pragmatic improvement over standard BO approaches, even on traditional computing hardware. Figure 5cii shows the total integrated complexity for a trial of  $n$  samples. The curve defines a threshold for the SSP size after which it is more efficient to use a GP implementation of BO. The time to compute the next sample point for SSP-BO is consistent with the predicted complexity from an asymptotic analysis—it remains constant regardless of the sample number. By adopting SSP-BO, we reduce the total time spent planning in our experiments by up to 205 × (see Table 3). Similarly, SSP-BO’s memory complexity is constant in  $t$ , in contrast with the empirical results from GP-BO (Supplementary Fig. 1) and the BoTorch (Fig. 6) algorithms. For SVGP, memory grows

**Table 3 | Total cumulative time spent planning sampling actions**

|                     | Total Time (sec ± s.e.) |                    | Improvement |
|---------------------|-------------------------|--------------------|-------------|
|                     | Exact GP                | SSP-BO (ours)      |             |
| Himmelblau          | 4.44 ± 1.52             | <b>1.44 ± 0.08</b> | 3.1 ×       |
| Goldstein-Price     | 6.54 ± 0.99             | <b>1.49 ± 0.06</b> | 4.4 ×       |
| Branin-Hoo          | 7.19 ± 1.58             | <b>1.28 ± 0.12</b> | 5.6 ×       |
| RNA Inverse Folding | 701.96 ± 2.65           | <b>3.42 ± 0.04</b> | 205.5 ×     |
| Pest Control        | 527.17 ± 6.18           | <b>2.99 ± 0.06</b> | 176.3 ×     |
| Box Reach           | 148.34 ± 9.20           | <b>0.90 ± 0.02</b> | 164.8 ×     |
| Belt Stretch        | 128.87 ± 2.50           | <b>1.45 ± 0.03</b> | 88.9 ×      |
| PSR Exploration     | 267.28 ± 18.96          | <b>4.49 ± 0.28</b> | 59.5 ×      |

The Exact GP baseline is GP (Matérn) for the standard target functions, GP (TO), the fastest baseline, for the combinatorial tasks, and BoTorch (GPU) for the robotics tasks. There is at least a 3 × improvement over baseline on toy problems, with as much as 200 × improvement over state-of-the-art for the more complicated tasks.

until the number of samples is greater than the maximum number of inducing points, at which point it becomes constant like SSP-BO. Overall, SSP-BO achieves efficiency gains by adopting VSA computing, without loss in performance.

## Discussion

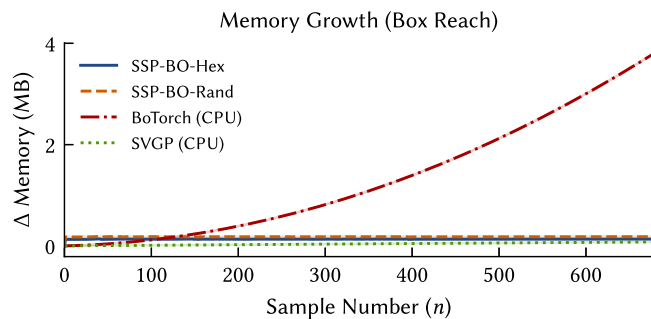
We present a BO algorithm, SSP-BO, which leverages VSA-based generalization of kernel embeddings and is the first BO method in which sample selection is demonstrated on neuromorphic hardware. The VSA embeddings enable us to retain the computational benefits of RFF-based GP approximations while extending BO to discrete, mixed, and structured data via hybrid symbolic-continuous representations.

SSP-BO delivers substantial computational gains, achieving constant-time and constant-memory sample selection with respect to the number of prior observations. We note that the embedding dimensionality  $d$  required to maintain a given error bound may grow with the number of observations for certain function classes. In many applications, including neuromorphic settings,  $d$  is fixed by hardware capacity, and one accepts a trade-off between representational fidelity and constant-time performance.

Empirically, we observe reductions of ~3 × to 200 × in total sample selection time, and at least a 30 × reduction in power consumption when translated to neuromorphic hardware. Memory consumption likewise favors SSP-BO, both empirically and asymptotically. These advantages grow with the number of observations, as standard GP costs scale with  $n$  while SSP-BO’s remain fixed.

A range of prior work has addressed compositional kernel construction for structured or discrete data. <sup>67</sup> proposed an algebra of tensor products and sums over kernels to construct kernels over discrete structures. <sup>68</sup> extended this to diffusion kernels, a generalized method of constructing kernels for data structures. These algebraic constructions underpin the automated identification of kernel functions as explored in the Automatic Statistician line of research<sup>7,8,61</sup>. These methods, while powerful, suffer from high computational cost due to  $n$ -wise comparisons over discrete domains.

More recently, BO in structured or combinatorial domains has used high-dimensional latent embeddings of discrete data. For example,<sup>64</sup> and<sup>100</sup> project combinatorial spaces into learned latent spaces using diffusion kernels, decoding optimal points via learned neural decoders.<sup>109</sup> instead reparameterize acquisition functions over probability distributions on discrete spaces. These approaches allow standard BO machinery to operate over complex input domains, but often require learned encoder-decoder pairs, and repeated nested optimizations over discrete and continuous components.



**Fig. 6 | Memory complexity growth for SSP-BO, BoTorch, and SVGP.** As with the standard GP implementations, BoTorch has a quadratic growth in memory consumption, while SSP-BO and SVGP remain constant. It takes ~106 samples on this 12 dimensional problem before BoTorch exceeds the memory complexity of SSP-BO. Data logged using the Heapy python memory profiler, data is from 10 trials, shaded regions represent 95% confidence intervals (not visible on graph, since memory utilization doesn't have large fluctuations).

Existing work on kernel approximation via RFFs demonstrates that additive kernels can be efficiently approximated by concatenating the feature maps for each component kernel<sup>72</sup>. However, composing kernels via multiplication requires tensor products of features, causing dimensionality to grow multiplicatively<sup>10</sup>. While low-rank tensor decompositions can alleviate this<sup>110</sup>, these methods require additional training.

In SSP-BO, we decode a VSA representation to obtain a data point, evaluate the objective function, and update the acquisition function with the re-encoded sample. Like prior latent-space BO methods,<sup>64,109</sup> SSP-BO performs acquisition updates in a latent space. However, unlike these approaches, it avoids learned representations entirely.

This design has several advantages. First, it yields interpretable embeddings: each component of the representation corresponds to a defined algebraic structure, not an opaque neural transformation. Second, unlike learned encoders, VSAs have established theory regarding their representational capacity (see section “Representational Capacity and Scaling Properties”), offering stronger guarantees about generalization and expressivity. Moreover, many structured kernels—such as those used in graph and string domains<sup>58,59</sup>—can be reformulated using VSAs. This formulation provides significant flexibility while keeping resource demands bounded. Because SSP-BO operates entirely in the embedding space, it avoids costly pairwise kernel computations and eliminates the need for multi-stage optimization pipelines. This makes SSP-BO especially attractive for scaling BO to complex, symbolic, or hybrid domains under finite-resource constraints.

Our method extends the applicability of kernel embeddings in several key ways, by providing an algebraic framework that realizes it within fixed computational resources, complementing standard kernel theory. First, we offer a compositional, dimensionality-preserving language for kernel construction, compatible with resource-constrained systems such as neuromorphic platforms. Second, the VSA framework naturally represents mixed discrete-continuous data in a unified space. While other methods require separate passes over different modalities and compute  $n$ -wise comparisons, our compositional algebra enables single-loop optimization across structured and hybrid domains. Decomposition only occurs during decoding, after a sample is chosen and not during optimization. We exploit this to apply a single optimization strategy across global function optimization (continuous spaces), combinatorial/graph optimization (discrete spaces), and robotics problems (mixed discrete-continuous spaces).

Finally, the nature of our algebra affords a degree of interpretability that is absent in learned encoder-decoder approaches [e.g.,<sup>61,63</sup>]. Because the feature representations are explicitly constructed and

algebraically manipulable, we can build, modify, or update them directly—composing new embeddings from atomic elements, shifting substructures, or rebinding values in place. This expressivity has been used in prior work on localization and mapping in spiking neural networks<sup>111,112</sup>.

This is the first work to implement part of a BO loop—sample selection—on neuromorphic hardware. Previous efforts like LavaBO<sup>113,114</sup> implement BO-compatible GPs, but only on CPUs, with future neuromorphic deployment intended. Likewise, earlier work on SSP-BO presented Neuromorphic-compatible optimization but was tested on a CPU<sup>115</sup>. In contrast, here we test our method on both SpiNNaker and Loihi 2.

Several efforts have explored accelerating GP inference using other specialized hardware. In particular, Cholesky decomposition and matrix-vector operations have been implemented on FPGA<sup>116,117</sup> and ASIC<sup>118</sup> platforms. However, these works focus on low-level GP inference rather than addressing the overall scaling limitations of GP-based BO or its application to high-dimensional action spaces.

Reducing BO's compute and memory demands introduces tradeoffs in approximation accuracy. However, as discussed in section “Representational Capacity and Scaling Properties”, the representational power of VSA bundles and Fourier embeddings is well-studied. For example, decoding error scales linearly with the number of bundled variables, indicating that while embedding size must grow with data complexity, the scaling remains tractable.

We note here that the HRR operations could, in principle, be combined with traditional RFFs to approximate compositional kernels. Future work could explore this. Moreover, the action spaces explored here are only a subset of what VSAs can represent. VSAs have been used to represent hash tables<sup>119</sup>, binary trees<sup>85</sup>, knowledge graphs<sup>87</sup>, words<sup>120</sup>, grammars<sup>121</sup>, and even images<sup>122</sup>. These results justify further exploration of SSP-BO in structured optimization.

Two major directions for future research are handling concept drift and enabling parameter adaptation within the BLR model. Concept drift could be addressed by introducing decay mechanisms into the BLR updates (Equation ??) to emphasize recent samples. Another promising approach is context binding, allowing for multi-context models that switch or blend based on task inference.

We also conjecture that adaptive parameters—particularly the length scale  $\ell$  in Equation (7)—could improve posterior refinement during later stages of learning. While SSP-BO performs strongly across the test cases reported here (and the BoTorch baseline already leverages adaptive parameters), performance may plateau as the optimization progresses. Given the connection between HexSSPs and mammalian grid cells, it is noteworthy that biological systems exhibit spatial adaptation: grid cells rescale with environmental change<sup>123</sup>, and place cells reorganize during learning<sup>124</sup>. Adaptive spatial representations may be crucial for long-term, flexible learning.

More generally, it would be worth exploring how the behavior and neural circuits implied by our model map onto models of learning and exploration that are tightly coupled to hippocampal anatomy [ref. 125]. More broadly, because SSP-BO operates entirely within the VSA algebra, it can serve as a drop-in component for exploration under uncertainty in existing VSA-based cognitive architectures such as the models of decision making, memory, spatial mapping, and other domains that already use these representations.

A final direction involves incorporating non-stationarity. In recursive SSPs<sup>90</sup>, embedding parameters such as length scale or phase matrices vary across space. This offers increased representational capacity and spatially dependent resolution. While conceptually promising, such embeddings remain underexplored and introduce additional complexity to encoding and decoding. Developing principled learning algorithms or heuristics for constructing recursive SSPs tailored to BO is an open challenge.

Bounded, or at least slowly-growing, resource requirements are vital to continually learning systems.<sup>126</sup> identified a number of biological mechanisms to support lifelong learning, including context-dependent perception and gating. Whether the algebraic nature of VSAs can support, or perhaps is a high-level description of, such mechanisms is an open point of investigation.

While primarily focused on the pragmatic benefits of SSP-BO in the context of robotics and reinforcement learning, this work closes an important explanatory gap in cognitive science. Specifically, while prior work proposed that compositional kernels are important for understanding how humans can learn with few samples, a neural implementation of these kernels was lacking. In this work, we have shown that compositional methods from cognitive science can provide compositional kernels without exploding the dimensionality of the representation. Because algebraic statements in VSAs can be converted into neural networks, and because we have constructed complex VSA representations from neural models of continuous and discrete spaces, we propose that VSAs close the gap between rule-like, compositional methods and neural implementations. A more detailed discussion of this point is in Supplementary Note 7, “Neuroscience and Cognition.”

## Methods

This section provides additional derivations for VSA-based kernel approximation, along with details on baseline methods and experimental setup. We first review Hexagonal Spatial Semantic Pointers [HexSSP;<sup>36</sup>], a structured projection of continuous data inspired by grid-cell neural representations<sup>127</sup>. We then elaborate on how VSA encodings support approximate compositional kernel construction. Decoding methods for extracting valid actions from optimization outputs are described, followed by a discussion of the representational capacity of VSA encodings. We review the baseline BO methods, highlighting key differences between RFF-based BO and our SSP-BO approach. We also detail the objective functions from the original robotics tasks. Lastly, we describe the recurrent spiking neural network used for sample selection in SSP-BO on neuromorphic hardware.

### Hexagonal SSPs

Hexagonal SSPs (Hex-SSPs) are a high-dimensional embedding of continuous-valued data in the HRR framework<sup>36</sup>. The construction of these embeddings can be thought of as an additive combination of hexagonally-tiled kernels, each defined by grids of different scales and orientations. Algorithm 2 shows the steps for constructing Hex-SSPs.

**Algorithm 2.** Hexagonal SSP Phase Matrix Generator.  $m$  is the dimensionality of the data being encoded;  $\mathcal{H} \subseteq \mathbb{R}$  are scaling values;  $\mathcal{R}$  are  $m$ -dimensional rotation matrices, and  $\mathbf{V}$  are structured phase matrices (see text below for details).

```

1: procedure HEX-SSP( $m, \mathcal{H}, \mathcal{R}$ )
2:    $\mathbf{V} \leftarrow [\mathbf{v}_1 \dots \mathbf{v}_{m+1}]^T \triangleright$  Vertices of a regular  $m$ -simplex
3:    $\mathbf{A} \leftarrow \text{stack}(\{h_i \mathbf{V} \mathbf{R}_i \mid h_i \in \mathcal{H}, \mathbf{R}_i \in \mathcal{R}, i = 1, \dots, G\})$ 
4:   return  $\mathbf{A}$ 
5: end procedure

```

Hexagonal SSP phase matrices are constructed from a regular  $m$ -simplex in an  $m$ -dimensional space. The simplex is determined by minimizing the relation  $\sum_{i=1}^{m+1} \sum_{j=1, i \neq j}^{m+1} \mathbf{v}_i \cdot \mathbf{v}_j$ , where  $\mathbf{v}_i, \mathbf{v}_j$  are unit vectors that make up the simplex. These vectors form the rows of an initial  $(m+1) \times m$  phase matrix,  $\mathbf{V}$ . We can specify the kernel function,  $k(\mathbf{x}, \mathbf{x}')$  that is approximated by the dot product between two SSPs,  $\phi(\mathbf{x})$  and  $\phi(\mathbf{x}')$ , by specifying the phase matrix of the encoding.

As a concrete example, consider the phase matrix for an SSP encoding of two-dimensional variables. The simplest simplex is the 2-simplex, formed from an equilateral triangle with three vertex vectors,  $\mathbf{v}_j$ , that are  $120^\circ$  apart. With these vectors defining the initial phase

matrix,  $\mathbf{V}$ , the resulting kernel function will have a hexagonally tiled pattern (the leftmost plot of Fig. 7a). This is due to the interference pattern of plane waves defined by the vectors  $\mathbf{v}_j$ . As the dimensionality of a Hexagonal SSP is increased (moving rightward and downward in Fig. 7a) more simplices are added, resulting in an interference pattern that enables Hex-SSPs to approximate a smooth kernel, and do so better than Rand-SSPs (Fig. 7b), which approximate a sinc function. While the sinc kernel is a positive definite function, it is not a probability density as it takes on negative values. However, it is still usable in kernel machines.

### Kernel approximations with VSAs

**Kernels over discrete encodings.** Discrete symbols (e.g., concepts, categories, slots) are represented in a high-dimensional vector space (see Equation (1)). The dot product between symbols represented in this fashion induces a kernel. This is a smoothly varying function with a relatively wide bandwidth, potentially inducing high similarity between randomly selected symbols. However, because we are randomly selecting points from a high-dimensional hypersphere, the likelihood of two vectors being similar is relatively low. The distribution of the dot product,  $\kappa = \phi(s) \cdot \phi(s')$ , between two randomly selected vectors,  $\phi(s), \phi(s')$ , representing the symbols  $s, s' \in \mathcal{S}$  is known to be<sup>128,129</sup>:

$$p(K = \kappa) = \frac{(1 - \kappa^2)^{\frac{d-1}{2}}}{\text{Beta}(\frac{1}{2}, \frac{d-1}{2})}. \quad (21)$$

The expected value of the dot product between any two such vectors is zero, with a variance inversely proportional to the dimensionality of the representation,  $\text{var}(K) = \frac{1}{d}$ . This means that noise induced by similarity between two different symbols can be controlled by increasing the representational dimension. The implication of this is that we can understand the kernel over discrete symbols to be:

$$k(s, s') = \begin{cases} 1 & s = s' \\ \kappa \sim p(K) & \text{otherwise} \end{cases} \quad (22)$$

Equation (22) is a useful formulation when analyzing the effect of cross-talk when taking the dot product between vector representations that are bundles of other vectors. By binding data vectors with symbol-vectors that specify the slot the data fills, one can make cross-talk terms arbitrarily small, for sufficiently high-dimensional vectors.

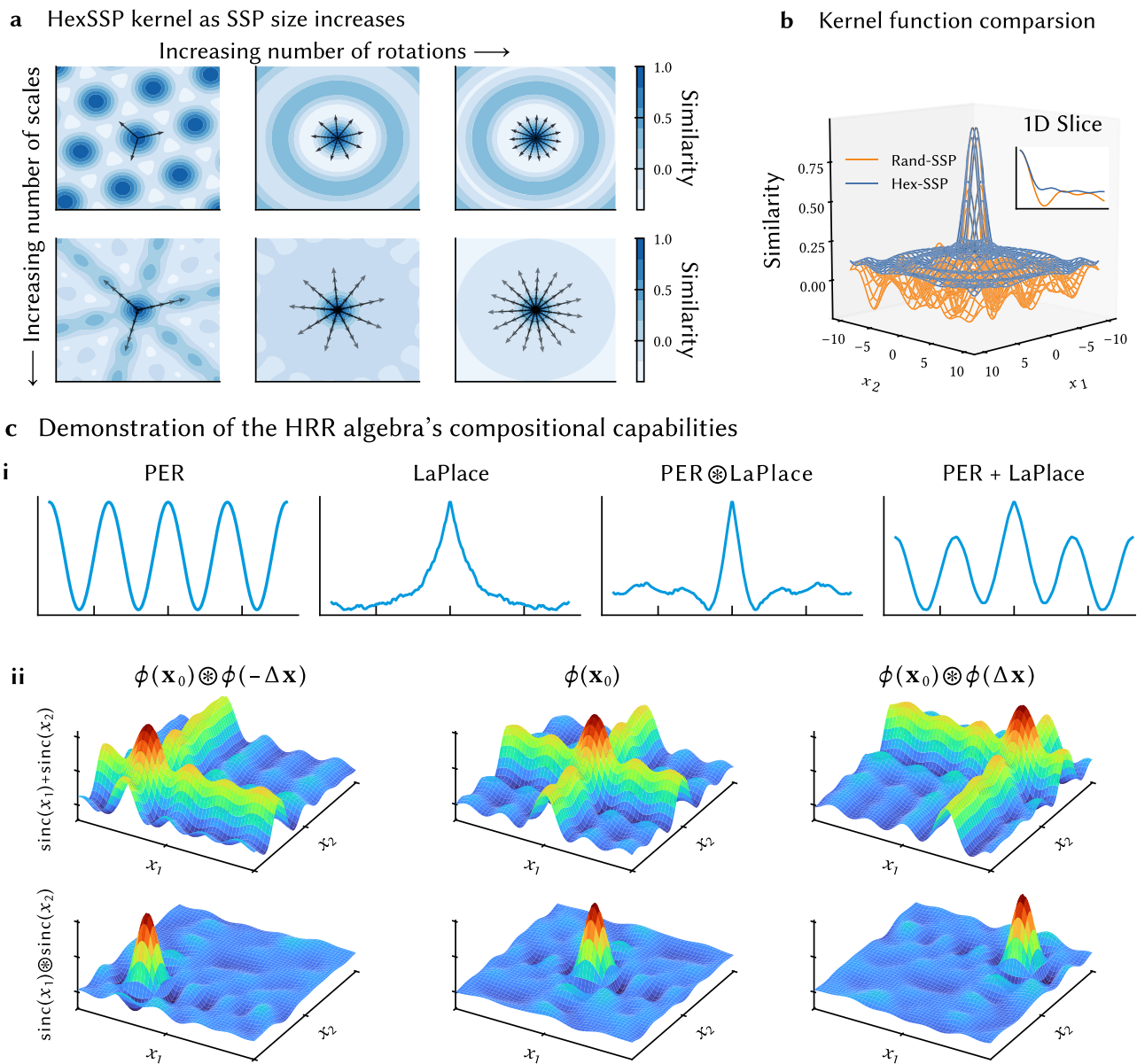
**Kernels over continuous encodings.** The dot product between SSPs is a kernel function,  $k(\mathbf{x}, \mathbf{y}) = k(\mathbf{x} - \mathbf{y}) = \phi(\mathbf{x}) \cdot \phi(\mathbf{y})$ . In general, the kernel's shape is dependent on the sampling distribution of the phase matrix components—in particular, it resembles the function whose power spectral density is the sampling distribution<sup>9,27</sup>. Suppose that the columns of the phase matrix are independent and identically distributed, sampled from a distribution  $P$  with probability density function  $p(\mathbf{a})$ , then the expected kernel is

$$\mathbb{E}_{\mathbf{A} \sim P}[\phi(\mathbf{x}) \cdot \phi(\mathbf{x} + \Delta \mathbf{x})] = \mathbb{E}_{\mathbf{A} \sim P} \left[ \frac{1}{d} e^{i \mathbf{A} \mathbf{x} / \ell} \cdot e^{i \mathbf{A} (\mathbf{x} + \Delta \mathbf{x}) / \ell} \right], \quad (23)$$

$$= \mathbb{E}_{\mathbf{A} \sim P} \left[ \frac{1}{d} \sum_{j=1}^d e^{i \mathbf{a}_j^T \Delta \mathbf{x} / \ell} \right], \quad (24)$$

$$= \int_{\mathbb{R}^n} e^{i \mathbf{a}^T \Delta \mathbf{x} / \ell} p(\mathbf{a}) d\mathbf{a}, \quad (25)$$

obtained using Parseval's theorem and the law of the unconscious statistician.



**Fig. 7 | SSP kernel functions and kernel composition.** **a** Kernel function induced by Hex-SSPs representing points in a 2D space as a heat map. The initial 3-component simplex (top left) produces hexagonal interference patterns like those found in grid cell neurons. Adding rotated simplices of the same scale (top right) produces a surface with a central peak surrounded by rings. Many differently-scaled simplices with the same orientation (bottom left) result in a starburst pattern. Adding many rotated and scaled simplices (bottom right) reduces the magnitude of the surrounding rings and increases smoothness and robustness to noise. **b** Rand-SSPs result in a sinc kernel function, while Hex-SSPs produce a sum of sinc functions (Equation (26)), making local maxima and minima are more shallow.

**c** Demonstration of the HRR algebra's compositional capabilities. **(i)** Shows the addition and multiplication of one dimensional kernels using periodic and LaPlace kernels encoded as SSPs. **(ii)** Shows that data encoded using additive and multiplicative kernel embeddings can be updated post-embedding. The central column shows the similarity between points in the domain and the original encoded point,  $\mathbf{x}_0$ . The left column shows the origin point shifted by  $-\Delta \mathbf{x}$ , a shift that is executed after encoding into the feature space using the binding operation of the HRR. The right column shows the same shift in the similarity map but for  $\Delta \mathbf{x}$ . A benefit of using VSAs is the ability to manipulate and update values after they have been projected into the high-dimensional space.

**HexSSP kernel.** The complete phase matrix  $\mathbf{A}$  used to construct Hex-SSPs is composed of multiple rotations and scalings of  $\mathbf{V}$ .  $\mathbf{V}$  is constructed in 2D by drawing vectors from  $\sim \text{Uniform}[-r, r]^2$ , and from each vector computing the two additional vectors needed to form a simplex. This changes the kernel function. For ease of illustration we consider continuous variables  $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^{m=2}$  and let  $\mathbf{x}' = \mathbf{x} + \Delta \mathbf{x}$ . The expectation of the resulting kernel function is given by Equation (26) below. It is a sum of sinc functions of the difference between the query

points,  $\Delta \mathbf{x}$ , projected onto the standard simplex.

$$\begin{aligned} \mathbb{E}[k_{\text{Hex-SSP}}(\mathbf{x}, \mathbf{x} + \Delta \mathbf{x})] &\propto (\text{sinc}(\Delta x_1) \text{sinc}(\Delta x_2)) \\ &+ \text{sinc}\left(-\frac{1}{2}\Delta x_1 + \frac{\sqrt{3}}{2}\Delta x_2\right) \text{sinc}\left(-\frac{\sqrt{3}}{2}\Delta x_1 - \frac{1}{2}\Delta x_2\right) \\ &+ \text{sinc}\left(-\frac{1}{2}\Delta x_1 - \frac{\sqrt{3}}{2}\Delta x_2\right) \text{sinc}\left(\frac{\sqrt{3}}{2}\Delta x_1 - \frac{1}{2}\Delta x_2\right) \end{aligned} \quad (26)$$

The rotations and scalings are introduced to explain the activity of neurons whose firing patterns have different grid sizes and orientations. However, an interesting pattern arises as more rotations and

scales are added to the representation: the similarity function becomes smoother, with one centralized peak and more shallow local optima, as shown in Fig. 7a. Furthermore, Hex-SSPs are more robust to noise and can be more accurately encoded in spiking neural networks via grid cells<sup>36</sup>.

**Data structures as approximate compositional kernels.** Core to VSA representations is the ability to construct data structures out of bundling and binding operations, given a suite of symbols and vector embeddings of numerical data. In section “Vector Symbolic Algebra (VSA)”, we described how one can represent sets, graphs, and trajectories. VSAs have also been used to represent other data structures in past work, including binary trees<sup>130</sup>, stacks<sup>131</sup>, n-gram statistics<sup>32</sup>, images<sup>122,132</sup>, sentences<sup>133</sup> and knowledge graphs<sup>87</sup>. A collection of permutation-based VSA encodings are surveyed by ref. 85, section IV, including stacks, finite state automata, and the directed graphs.

What is important about these representations is that their similarity, as computed by the vector dot product, captures similarity between data structures that is controlled by their structure. In other words, using vector representations as a base set, we can understand the HRR algebra as a language for composing kernels. In the style of<sup>8</sup>, the algebra of HRRs permits the addition and multiplication of kernels, using the bundling and binding operators.

It has already established that the dot product between symbols (discrete or continuous) can be interpreted as kernel functions between the encoded data<sup>12,27–29</sup>. In fact, in this framework, there is no meaningful distinction between discrete, or continuous data. By building up data structures out of these elements, using the circular convolution algebra that Plate proposed, we will show that dot product similarity between data structures induces kernel functions for the data structures as well. This complements and expands on past work connecting VSAs to kernel approximation<sup>28</sup> and work on compositionality of functions embedded in reproducing kernel Hilbert spaces, using the FHRR algebra<sup>29</sup>.

We appeal to the method of Random Fourier Features, originated by Rahimi and Recht<sup>9</sup>, to confirm that we can shape the kernels to our liking, as demonstrated by refs. 29, 134, and 36. What we show next is that binding and bundling can be used to construct kernels from elementary components. However, we note that, by virtue of the finite representations in neural networks, all kernels will necessarily be bandlimited kernels.

Given that the dot product between base vectors induces a kernel, we can use binding to multiply kernels. We show the connection between bundling and kernel multiplication through operation in the Fourier domain:

$$\mathcal{F}\{\phi_X(\mathbf{x}) \otimes \phi_Y(\mathbf{y}) \cdot \phi_X(\mathbf{x}') \otimes \phi_Y(\mathbf{y}')\} \\ = e^{iA_X \mathbf{x}} \odot e^{iA_Y \mathbf{y}} (e^{iA_X \mathbf{x}'} \odot e^{iA_Y \mathbf{y}'})^\dagger \quad (27)$$

$$= D(e^{iA_X \mathbf{x}}) e^{iA_Y \mathbf{y}} (e^{-iA_Y \mathbf{y}'} )^\dagger D(e^{-iA_X \mathbf{x}'}) \quad (28)$$

$$\approx D(e^{iA_X \mathbf{x}}) k_Y(\mathbf{y}, \mathbf{y}') D(e^{-iA_X \mathbf{x}'}) \quad (29)$$

$$= e^{iA_X \mathbf{x}} (e^{-iA_X \mathbf{x}'} )^\dagger \odot k_Y(\mathbf{y}, \mathbf{y}') \quad (30)$$

$$\approx k_X(\mathbf{x}, \mathbf{x}') k_Y(\mathbf{y}, \mathbf{y}') \quad (31)$$

where  $D(\cdot)$  signifies the diagonal matrix of its argument and  $\dagger$  is the conjugate transpose. In equations (29) and (31) we use the approximation sign to indicate we are approximating a kernel function with a finite-dimensional dot product.

To construct additive kernels, we can use bundling of vector-encodings of data. This connection is trivial to demonstrate, as the dot product distributes over the vector sum:

$$(\phi_X(\mathbf{x}) + \phi_Y(\mathbf{y})) \cdot (\phi_X(\mathbf{x}') + \phi_Y(\mathbf{y}')) \\ = \phi_X(\mathbf{x}) \cdot \phi_X(\mathbf{x}') + \phi_X(\mathbf{x}) \cdot \phi_Y(\mathbf{y}') \\ + \phi_Y(\mathbf{y}) \cdot \phi_X(\mathbf{x}') + \phi_Y(\mathbf{y}) \cdot \phi_Y(\mathbf{y}') \quad (32)$$

$$\approx k_X(\mathbf{x}, \mathbf{x}') + \phi_X(\mathbf{x}) \cdot \phi_Y(\mathbf{y}') \\ + \phi_Y(\mathbf{y}) \cdot \phi_X(\mathbf{x}') + k_Y(\mathbf{y}, \mathbf{y}') \quad (33)$$

$$\approx k_X(\mathbf{x}, \mathbf{x}') + k_Y(\mathbf{y}, \mathbf{y}') + 2\varepsilon \quad (34)$$

where  $2\varepsilon$  is the cross-talk noise,  $\phi_X(\mathbf{x}) \cdot \phi_Y(\mathbf{y}') + \phi_X(\mathbf{x}') \cdot \phi_Y(\mathbf{y})$ . Examples of multiplicative and additive kernels are given in Fig. 7ci in one dimension, and in two dimensions in Fig. 7cii (multiplicative in bottom row and additive in top row). There are pathological cases where the cross-talk noise is large, e.g.,  $\mathbf{x} = \mathbf{x}' = \mathbf{y} = \mathbf{y}' \approx \mathbf{0}$ . Cross-talk for additive kernels can be mitigated by using concatenation instead of bundling, as has been employed in RFF-based approaches [e.g.,<sup>72</sup>]. However, concatenation does not preserve dimensionality, and in the spatial domain, concatenation destroys the ability to apply binding techniques to vector representations.

Fortunately, in the HRR algebra, cross-talk noise can be controlled through the use of slot-filler representations. A bundle of the form  $\phi_X(\mathbf{x}) + \phi_Y(\mathbf{y})$  may be re-written as  $G_X \otimes \phi_X(\mathbf{x}) + G_Y \otimes \phi_Y(\mathbf{y})$  where  $G_X$  and  $G_Y$  are randomly generated discrete vectors that denote the role the data is playing in the sum. This ensures that variables  $\mathbf{x}$  and  $\mathbf{y}$  are projected onto different parts of the high-dimensional feature space. Consequently, the cross-talk noise is scaled by the similarity between the slot vectors,  $G_X$  and  $G_Y$ , which should be very small. Bundling, after binding to a slot, lets us approximate additive kernels. Figure 7c illustrates the ability to compose kernels using the HRR algebra.

Since we know that positive definite kernels are closed under addition and multiplication<sup>8,66</sup>, and that our algebra can implement kernel multiplication and addition, we can represent compositional kernels, such as those produced by the kernel description grammar used by the Automatic Statistician<sup>8</sup>. These results also imply that, provided we have meaningful data projections for the individual elements, any data structure that is constructed from bundling and binding operations implicitly creates a bandlimited kernel function for the structured data.

## Feature decoding

The BO routine produces an optimal vector,  $\theta^*$ , in the feature space. However, the objective function requires sample points from its domain. In systems where VSA embeddings are used throughout (e.g., a robot whose motor system operates directly on VSA-encoded commands  $\theta^*$  may be used directly. In most cases, including all tasks in this paper, we must decode the optimal point in the domain,  $\mathbf{x}^* \in \mathcal{X}$ , from  $\theta^*$ .

Decoding SSPs through optimization, as in ref. 72, is challenging because kernels induced by the dot product between Hex-SSPs have an infinite number of local maxima, increasing the complexity of decoding. To decode an SSP  $\theta^*$ , we first coarsely sample  $\mathcal{X}$  and encode the samples as SSPs. Samples are spaced by  $2/l_i$  along each axis,  $i \in \{1, \dots, m\}$ , where  $l_i$  is the length scale used in the SSP encoding of the  $i^{\text{th}}$  axis. The SSP with the highest similarity to  $\theta^*$  is used as the initial point of a gradient-based optimization, whose objective is to find  $\mathbf{x}_s$  such that  $\phi_{\mathcal{X}}(\mathbf{x}_s)$  has maximal similarity with the optimal SSP,  $\theta^*$ . The corresponding point  $\mathbf{x}_s$  is selected as the decoded point. Since there is a connection between the VSA representations we use and probability distributions [as we discuss in refs. 30, 95], this decoding process is akin to sampling a maximum a posteriori prediction.

One could alternatively decode using a neural network, as done in ref. 89. The downsides of this approach are that the required training data grows with the input domain dimensionality, and the network must be retrained for different length scales. However, the training data is easy to generate and network training can be done prior to deployment of the optimization algorithm.

For the combinatorial and graph VSA embeddings (Equation (3)), we decode by unbinding each slot vector and then finding the category vectors that maximize similarity:  $\arg \max(\theta^* \odot \phi_S(s_i)) \cdot \phi_{V_i}(v_{i,j})$  for  $i = 1$  to the total number of categories. Likewise, with trajectory embeddings, we can use unbinding to decode the proposed set of waypoints:  $\arg \max_{\mathbf{x}}(\theta^* \odot \phi_T(t_i)) \cdot \phi_X(\mathbf{x})$ .

### Representational capacity and scaling properties

One of the key advantages of SSP-BO is that its runtime and memory requirements remain constant with respect to the number of collected observations,  $n$ . Specifically, it avoids the  $\mathcal{O}(n^3)$  time and  $\mathcal{O}(n^2)$  memory costs of standard GP surrogate models, replacing them with  $\mathcal{O}(d^2)$  and  $\mathcal{O}(d^2)$ , respectively, where  $d$  is the dimensionality of the vector embeddings. The  $\mathcal{O}(d^2)$  time complexity arises from using the Sherman-Morrison formula to perform rank-one updates to  $\Sigma_n$  (Equation (15)). Since  $d$  is a fixed, user-selected parameter that can be chosen based on the available computational budget, and the theoretical bounds reviewed later, SSP-BO is well suited to deployment on neuromorphic hardware and other resource-constrained platforms.

There is a well-established body of theory on the representational capacity of VSAs that we can draw on to understand how the choice of  $d$  affects performance. In short, both theory and empirical results show that the required dimensionality scales efficiently: linearly with the number of bundled items and only logarithmically with dictionary size for discrete representations (to obtain constant decoding error); and linearly with the data dimensionality for continuous representations (to obtain constant kernel approximation error). This means that relatively modest values of  $d$  are sufficient for accurate decoding and function approximation in practice. Additionally, unlike learned latent embeddings—where capacity limits and scaling behavior are often opaque—VSA-based methods offer transparent, analyzable tradeoffs between dimensionality, accuracy, and generalization.

**Discrete Representations:** Discrete symbols are encoded according to Equation (1). For bundles of  $K$  discrete vectors drawn from a dictionary of size  $m$ ,<sup>83</sup> gives a bound on the dimensionality  $d$  needed to decode with error probability  $q$ :

$$d \geq 3.16(K - 0.25) \log \left( \frac{m}{q^3} \right). \quad (35)$$

This bound confirms efficient scaling:  $d$  grows linearly with bundle size and only logarithmically with vocabulary size. Theoretical predictions have been validated in practice<sup>20,135</sup> and extended by ref. 79, who benchmarked multiple VSAs. FHRRs and sparse binary architectures consistently required the lowest dimensionalities to achieve high decoding accuracy—outperforming alternatives like Multiply-Add-Permute, Vector-derived Transform Binding, and Matrix Binding of Additive Terms. Note that while our embeddings live in the real domain, as in HRR, our approach to generating vectors is actually taken from FHRR (thus, unbinding is exact rather than approximate). As such, the representational scaling of FHRRs is most relevant.

**Continuous Representations:** The SSP embedding of continuous data is similar to that of RFF—in fact, SSPs can be obtained from a linear transform on RFFs with particular constraints. Thus, capacity results from the RFF literature are applicable to SSP-BO. For example, the error of approximating a kernel is less than  $\epsilon$  with high probability

when

$$d \geq \mathcal{O} \left( \frac{\log \text{diam}(\mathcal{X})}{\epsilon^2} \right), \quad (36)$$

where  $\text{diam}(\mathcal{X})$  is the diameter of the compact space that the values of  $\mathbf{x}$  occupy [<sup>136</sup>, Theorem 1].

When applying kernel ridge regression using RFFs, the required dimensionality for minimax-optimal learning rates is:

$$d = \mathcal{O}(\sqrt{N} \log d_\lambda) \quad (37)$$

where  $d_\lambda$  denotes the effective degrees of freedom of the kernel and  $N$  is the number of observations<sup>137,138</sup>. In other words, the required dimensionality grows sublinearly with data set size.<sup>139</sup> further showed that RFFs of size  $\mathcal{O}(\sqrt{N} \log N)$  can match the generalization bounds of exact kernel ridge regression. This further supports the practical efficiency of SSPs in learning tasks. Empirically, we evaluate the effect of  $d$  and data size  $m$  on the performance of SSP-BO in Supplementary Note 4.3, “Dimensionality Experiments”.

Since all VSA structures ultimately represent bundles of bound items, the representational and regression requirements interact. A safe and effective practice is to select  $d$  according to the maximum of the bounds given in Equation (35) and Equation (37). That said, we emphasize that the regression bounds are derived under conservative assumptions – namely, uniform function approximation over entire input domains. In practice, SSP-BO often targets local or structured objectives, which allows for significantly smaller  $d$  values than theory might suggest.

For example, in our own experiments, values of  $d$  as low as 25 yielded strong performance across a range of challenging tasks. This confirms that SSP-BO remains computationally efficient and practically effective even in constrained settings.

### Neuromorphic implementation for SSP-BO

We implemented the sample selection step of the SSP-BO algorithm on neuromorphic hardware platforms. Sample selection in SSP-BO is enabled on neuromorphic hardware by formulating the optimization directly in the smooth, high-dimensional embedding space, rather than in the typically non-smooth action space. This allows a spiking neural network to perform gradient ascent on the acquisition function by evolving its internal neural state over time. Figure 1b illustrates the recurrent network architecture used for this process: a population of spiking neurons encodes the embedding  $\phi$  nonlinearly, with linear decoding used to read out the current estimate of  $\phi$ . Recurrent connections within the population are trained to approximate the gradient ascent of the acquisition function, following the principles of the Neural Engineering Framework (NEF;<sup>140</sup>). After 2.5 seconds of simulation time, the optimized point  $\phi^*$  is decoded from the spiking population and used to update the surrogate model.

During each simulation window, the network evolves smoothly in the neural state space. When decoded back into the original low-dimensional action space, however, the resulting trajectory can exhibit jumps and a mixture of local and global search.

For the experiments reported on neuromorphic hardware, we use an SSP size of  $d = 55$ , which is lower than the  $d = 97$  used in Fig. 4a. The SpiNNaker implementation uses 385 leaky integrate-and-fire (LIF) neurons, while the Loihi 2 implementation used 440 ReLU-like neurons, enabled by exploiting graded spike functionality.

### Baseline implementations for BO

As a baseline for our standard test function experiments we use the algorithm of<sup>50</sup>, implemented using the scikit-learn Gaussian process package<sup>141</sup>, which we denote as GP (Matérn) or GP (sinc), depending on the choice of kernel. We also compare to approximate GP using RFFs,

adapted from <sup>142</sup>, which we denote as RFF-BO (see Supplementary Algorithm 3). This method uses the same acquisition function and BLR updates as SSP-BO (Equation (18) and Equation (16)), but differs in three ways, with the last two being the most significant.

- **Linear transform:** The RFFs are given by  $\frac{2\sigma^2}{d} \cos(A\mathbf{x})$ , where  $\sigma$  is fit using the initial samples. This could be obtained from an SSP constructed using  $A$  via  $\frac{2\sigma^2}{d} \text{Re } \mathcal{F}\{\phi(\mathbf{x})\}$ —in other words, it differs from an SSP by a linear transform and an additional scaling term.
- **Phase matrix generation:** The phase matrix elements,  $A$ , in the RFFs are sampled from  $\mathcal{N}(0, \frac{1}{\ell})$ , where  $\ell$  is optimized using the scikit-learn GP regressor with an RBF kernel over the initial samples. While in SSP-BO-Rand the elements are sampled from a uniform distribution and in SSP-BO-Hex they are generated using Algorithm 2.
- **Optimization domain:** In RFF-BO, the acquisition function is optimized over the data domain,  $\max \psi_n(\mathbf{x})$ . In contrast, SSP-BO optimizes in the embedding space  $\mathbf{x} \in \mathbb{R}^n$  over a continuous, compact domain and then decodes (Equation (19)).

An additional distinction is that RFF embeddings are not defined for the combinatorial, graph, or trajectory tasks. For the combinatorial tasks, we use the GP BO methods tested in ref. <sup>98</sup>. The algorithms use various kernels, acquisition functions, and optimization methods that operate over combinatorial spaces. The options for each method included in Fig. 5a are given in Supplementary Table II.

For the robotics experiments, we use a state-of-the-art Bayesian optimization library called BoTorch<sup>143</sup>. BoTorch offers a variety of options for Gaussian processes (built on top of GPyTorch<sup>144</sup>) and their kernels. Additionally, it allows for kernel hyperparameters to be fit automatically using marginal likelihood, as described in Section V-A in ref. <sup>145</sup>. Many recent BO methods are built on top of BoTorch and use the standard library as a baseline <sup>100,146–148</sup>.

One of the most widely used families of kernels is the Matérn kernel family. Within this kernel family, the Radial Basis Function (RBF) kernel (also known as the Squared Exponential kernel) has been shown to perform well on a range of applications in robot locomotion and manipulation<sup>149–154</sup>. Hence, we use the RBF kernel in our experiments. We use UCB as the acquisition function for BO, since it is one of the most widely used options with both good practical performance and theoretical guarantees for regret bounds<sup>94</sup>. We compare against both exact GP inference—using BoTorch to compute Gaussian posteriors – and SVGP, a sparse variational GP model<sup>155</sup> with a fixed number of inducing points, initialized following <sup>156</sup>. The number of inducing points is chosen to match the memory usage of SSP-BO.

## Experimental setup and environments

We present three types of tasks, all of which are illustrated in Fig. 1c. The first task suite consists of a set of test functions commonly used for evaluating global optimization algorithms. The second evaluation suite consists of black-box optimization functions defined over multivariate discrete spaces (e.g., combinatorial variables, graphs). Functions from these first two sets have all been used as benchmarks in prior work. The third type of task contains robots and autonomous exploration tasks. Robot simulations were built using the PyBullet physics simulator<sup>157</sup>. All robotics tasks have a total path distance penalty term,  $p(\mathbf{x})$ , to encourage efficiency.

**Dual-arm robot manipulation.** In the first robot experiment, Box Reach, there are two robot arms that need to reach targets in a box (see Fig. 1c). The two arms have different sizes. The smaller arm is able to reach into smaller boxes, but not larger boxes. The other arm is larger and can reach into boxes of all heights, but moves slower. The reward function is based on the distances to the target from the closest arm at

each time step  $t$  (and the penalty term  $p(\mathbf{x})$ ):

$$\sum_{t=0}^T \exp\left\{-\min(\text{dist}_t(\text{goal}, \text{arm}_1), \text{dist}_t(\text{goal}, \text{arm}_2))^2/2\right\} - p(\mathbf{x}). \quad (38)$$

The control policy is a selection of two 3D waypoints, a mid-trajectory waypoint and an end-trajectory waypoint, for the end effector of each arm, which are then executed by an onboard operational space controller.

In the second robot experiment, Belt Stretch, the robots need to stretch a rubber belt onto a belt-drive mechanism (see the middle part of Fig. 1c). The robots need to learn to coordinate both arms while placing the rubber belt on the mechanism. The reward for the task is based on the distances between the current and desired belt pose. In our experiments, we express the desired belt pose by defining two target locations on each wheel of the drive mechanism and checking the distance from the four equally spaced locations (vertices) of the belt to these target locations on the wheels:  $\sum_{t=0}^T \exp\left\{-\sum_{k=1}^{K=4} \text{dist}_t(\text{belt\_vertex}_k, \text{target}_k)^2/2\right\} - p(\mathbf{x})$ . The search space is five waypoints per arm, executed by both arms simultaneously, for a total of ten 3D waypoints.

**Multi-robot exploration of permanently shadowed regions.** In our third robot environment, we have a swarm of  $n$  robots collaboratively exploring a Permanently Shadowed Region (PSR) in order to find a maximum density of subsurface water. Here the task is executed in two stages. In stage one, the swarm of robots explore the interior of the PSR to find the maximum estimated subsurface water density. They do this exploration under the constraints of solar power, and need to exit the PSR to recharge and share map data. In the second stage the PSR is entered by a robot that carries a drill, which goes to the location of maximum estimated subsurface water density, and collects a soil sample before exiting the PSR. The first stage of activity acts as the Bayesian optimization, learning a function before executing on the learned knowledge.

The reward function during the exploration stage is the maximum of the estimated water density observed on each robot path. The control action is a set of waypoint trajectories assigned to the robots in the swarm. We assume that the robots will reverse their trajectories to return to the crater rim to recharge in sunlight.

Observations of subsurface water were modeled on an approximation of a neutron spectrometer. Neutron spectrometer data can be modeled as Poisson-distributed counts driven by the abundance of water in the subsurface under the instrument. In this experiment we are using high count values as a proxy for high water abundance.

We created a water map using the method described in (author?) [<sup>158</sup>, Ch. 4]. We constructed a Voronoi map from 10 seed points uniformly sampled from the crater interior. Each seed point was assigned values sampled uniformly in the range [20, 140], mimicking values observed in the Mojave desert during NASA's Mojave Volatiles Prospector project. To prevent high-value locations from being generated outside of the PSR, a ring of low-value counts were added to the perimeter of the simulated PSR. We then convolved the Voronoi water map with a Gaussian blur of 7 pixels, to account for the blurring process of the field of view of the instrument. The above process creates a map from location to neutron counts,  $c(\mathbf{x})$ , independent of terrain geometry. Observed counts are then defined as follows:

$$c_{\text{obs}}(\mathbf{x}) = \max\{0, c(\mathbf{x}) \cdot (\text{height}_{\text{max}} - \text{height}(\mathbf{x})/\text{height}_{\text{max}})\} \quad (39)$$

To account for the instability of water abundance outside of a permanently shadowed region, the count map is scaled down to zero when the elevation of the terrain exceeds the maximum height of the crater.

## Data availability

The performance and timing data generated in this work have been deposited in a *Zenodo* repository at (<https://doi.org/10.5281/zenodo.19893169>) under a Creative Commons Attribution 4.0 license.

## Code availability

The SSP-BO code is publicly available at (<https://github.com/ctn-waterloo/ssp-bayesopt>) under an MIT license. The robotics experiments code is available at (<https://github.com/contactrika/bo-ssp>) under an MIT license.

## References

- Frankland, S. M. & Greene, J. D. Concepts and compositionality: in search of the brain's language of thought. *Annu. Rev. Psychol.* **71**, 273–303 (2020).
- Ito, T. et al. Compositional generalization through abstract representations in human and artificial neural networks. *Adv. neural Inf. Process. Syst.* **35**, 32225–32239 (2022).
- Kurth-Nelson, Z. et al. Replay and compositional computation. *Neuron* **111**, 454–469 (2023).
- Schwartenbeck, P. et al. Generative replay underlies compositional inference in the hippocampal-prefrontal circuit. *Cell* **186**, 4885–4897 (2023).
- Lake, B. M., Salakhutdinov, R. & Tenenbaum, J. B. Human-level concept learning through probabilistic program induction. *Science* **350**, 1332–1338 (2015).
- Smolensky, P., McCoy, R., Fernandez, R., Goldrick, M. & Gao, J. Neurocompositional computing: from the central paradox of cognition to a new generation of ai systems. *AI Mag.* **43**, 308–322 (2022).
- Schulz, E., Tenenbaum, J. B., Duvenaud, D., Speekenbrink, M. & Gershman, S. J. Compositional inductive biases in function learning. *Cogn. Psychol.* **99**, 44–79 (2017).
- Duvenaud, D., Lloyd, J., Grosse, R., Tenenbaum, J. & Zoubin, G. Structure discovery in nonparametric regression through compositional kernel search. In *International Conference on Machine Learning*, 1166–1174 (PMLR, 2013).
- Rahimi, A., Recht, B. et al. Random features for large-scale kernel machines. *NIPS* **3**, 5 (2007).
- Muandet, K., Fukumizu, K., Sriperumbudur, B. & Schölkopf, B. Kernel mean embedding of distributions: A review and beyond. *Found. Trends® Mach. Learn.* **10**, 1–141 (2017).
- Plate, T. A. *Distributed representations and nested compositional structure*. Ph.D. thesis, University of Toronto, Toronto, ON, Canada (1994).
- Plate, T. A. *Holographic Reduced Representation: Distributed Representation For Cognitive Structures* (CSLI Publications, Stanford, CA, USA, 2003).
- Gayler, R. W. Vector symbolic architectures answer Jackendoff's challenges for cognitive neuroscience. In *In Proceedings of the Joint International Conference on Cognitive Science (ICCS/ASCS'03)*, 133–138 (The University of New South Wales, 2003).
- Smolensky, P. Tensor product variable binding and the representation of symbolic structures in connectionist systems. *Artif. Intell.* **46**, 159–216 (1990).
- Kanerva, P. Binary spatter-coding of ordered k-tuples. In *International Conference on Artificial Neural Networks*. 869–873 (Springer, 1996).
- Gayler, R. W. et al. Multiplicative binding, representation operators, and analogy. *Advances in Analogy* **405** (1998).
- Rachkovskij, D. A. & Kussul, E. M. Binding and normalization of binary sparse distributed representations by context-dependent thinning. *Neural Comput.* **13**, 411–452 (2001).
- Levy, S. D. & Gayler, R. Vector symbolic architectures: A new building material for artificial general intelligence. In *Proceedings of the 2008 conference on artificial general intelligence 2008: Proceedings of the first AGI conference*, 414–418 (2008).
- Eliasmith, C. et al. A large-scale model of the functioning brain. *science* **338**, 1202–1205 (2012).
- Eliasmith, C. et al. *How to Build A Brain: A Neural Architecture For Biological Cognition*. (Oxford University Press, 2013).
- Hersche, M., Zeqiri, M., Benini, L., Sebastian, A. & Rahimi, A. A neuro-vector-symbolic architecture for solving Raven's Progressive Matrices. *Nat. Mach. Intell.* **5**, 363–375 (2023).
- Ganesan, A. et al. Learning with holographic reduced representations. *Adv. neural Inf. Process. Syst.* **34**, 25606–25620 (2021).
- Karunaratne, G. et al. Robust high-dimensional memory-augmented neural networks. *Nat. Commun.* **12**, 2468 (2021).
- Alam, M. M., Raff, E., Biderman, S., Oates, T. & Holt, J. Recasting self-attention with holographic reduced representations. In *International Conference on Machine Learning*, 490–507 (PMLR, 2023).
- Yeung, C. et al. Structure-aware attention based on vector symbolic architectures. <https://openreview.net/forum?id=zETOZg71WT> (2025).
- Plate, T. A. *Holographic Reduced Representation: Distributed representation for cognitive structures*, vol. 150 (CSLI Publications, Stanford, 2003).
- Voelker, A. R. A short letter on the dot product between rotated Fourier transforms. *arXiv preprint arXiv:2007.13462* (2020).
- Thomas, A., Dasgupta, S. & Rosing, T. A theoretical perspective on hyperdimensional computing. *J. Artif. Intell. Res.* **72**, 215–249 (2021).
- Frady, E. P., Kleyko, D., Kymn, C. J., Olshausen, B. A. & Sommer, F. T. Computing on functions using randomized vector representations. *arXiv preprint arXiv:2109.03429* (2021).
- Furlong, P. M. & Eliasmith, C. Modelling neural probabilistic computation using vector symbolic architectures. *Cogn. Neurodyn.* **18**, 1–24 (2023).
- Kleyko, D. et al. Principled neuromorphic reservoir computing. *Nat. Commun.* **16**, 640 (2025).
- Joshi, A., Halseth, J. T. & Kanerva, P. Language geometry using random indexing. In *Quantum Interaction: 10th International Conference, QI 2016, San Francisco, CA, USA*, **10**, 265–274 (Springer, 2017).
- Plate, T. A. et al. Holographic recurrent networks. *Advances in Neural Information Processing Systems*. **5** (1992).
- Frady, E. P., Kanerva, P. & Sommer, F. T. A framework for linking computations and rhythm-based timing patterns in neural firing, such as phase precession in hippocampal place cells. In *Proceedings of the Conference on Cognitive Computational Neuroscience* (2018).
- Komer, B., Stewart, T. C., Voelker, A. & Eliasmith, C. A neural representation of continuous space using fractional binding. In *CogSci*, **41**, 2038–2043 (2019).
- Dumont, N. S.-Y. & Eliasmith, C. *Accurate Representation For Spatial Cognition Using Grid Cells*. In *42nd Annual Meeting of the Cognitive Science Society*. Toronto, ON: Cognitive Science Society, 2367–2373 (2020).
- Dumont, N. S.-Y. *Symbols, Dynamics, and Maps: A Neurosymbolic Approach to Spatial Cognition*. Phd thesis, University of Waterloo, Waterloo, ON (2025).
- Krausse, S., Neftci, E., Sommer, F. T. & Renner, A. A grid cell-inspired structured vector algebra for cognitive maps. In *Proceedings of the 2025 Annual Neuro-Inspired Computational Elements Conference* (2025).
- Turchetta, M., Krause, A. & Trimpe, S. *Robust Model-free Reinforcement Learning With Multi-objective Bayesian Optimization*. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 10702–10708 (IEEE, 2020).

40. Imani, M. & Ghoreishi, S. F. Scalable inverse reinforcement learning through multifidelity Bayesian optimization. *IEEE Trans. neural Netw. Learn. Syst.* **33**, 4125–4132 (2021).
41. Hensman, J., Durrande, N. & Solin, A. Variational Fourier features for Gaussian processes. *J. Mach. Learn. Res.* **18**, 1–52 (2018).
42. Bourgault, F., Makarenko, A. A., Williams, S. B., Grocholsky, B. & Durrant-Whyte, H. F. Information based adaptive robotic exploration. In *IEEE/RSJ international conference on intelligent robots and systems*. 1540–1545 (IEEE, 2002).
43. Thompson, D. R. & Wettergreen, D. Intelligent maps for autonomous kilometer-scale science survey (2008).
44. Charrow, B., Liu, S., Kumar, V. & Michael, N. *Information-theoretic Mapping Using Cauchy-schwarz Quadratic Mutual Information*. In *2015 IEEE International Conference on Robotics and Automation (ICRA)*, 4791–4798 (IEEE, 2015).
45. Liao, T. et al. Data-efficient learning of morphology and controller for a microrobot. In *2019 International Conference on Robotics and Automation (ICRA)*, 2488–2494 (IEEE, 2019).
46. Frazier, P. I. & Wang, J. *Bayesian Optimization For Materials Design*. In *Information Science For Materials Discovery And Design*, 45–75 (Springer, 2015).
47. Wu, J. et al. Hyperparameter optimization for machine learning models based on Bayesian optimization. *J. Electron. Sci. Technol.* **17**, 26–40 (2019).
48. Singh, A., Krause, A., Guestrin, C., Kaiser, W. J. & Batalin, M. A. Efficient planning of informative paths for multiple robots. In *IJCAI*. **7**, 2204–2211 (2007).
49. Yang, K., Keat Gan, S. & Sukkarieh, S. A Gaussian process-based RRT planner for the exploration of an unknown and cluttered environment with a uav. *Adv. Robot.* **27**, 431–443 (2013).
50. Contal, E., Perchet, V. & Vayatis, N. Gaussian process optimization with mutual information. In *International Conference on Machine Learning*, 253–261 (PMLR, 2014).
51. Quinero-Candela, J. & Rasmussen, C. E. A unifying view of sparse approximate gaussian process regression. *J. Mach. Learn. Res.* **6**, 1939–1959 (2005).
52. Gal, Y. & Turner, R. Improving the Gaussian process sparse spectrum approximation by representing uncertainty in frequency inputs. In *International Conference on Machine Learning*, 655–664 (PMLR, 2015).
53. Little, D. Y.-J. & Sommer, F. T. Learning and exploration in action-perception loops. *Front. neural circuits* **7**, 37 (2013).
54. Tabib, W., Corah, M., Michael, N. & Whittaker, R. Computationally efficient information-theoretic exploration of pits and caves. In *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 3722–3727 (IEEE, 2016).
55. Marco, A. et al. Virtual vs. real: Trading off simulations and physical experiments in reinforcement learning with Bayesian optimization. In *IEEE International Conference on Robotics and Automation (ICRA)*, 1557–1563 (IEEE, 2017).
56. Cully, A., Clune, J., Tarapore, D. & Mouret, J.-B. Robots that can adapt like animals. *Nature* **521**, 503–507 (2015).
57. Rodriguez, D., Brandenburger, A. & Behnke, S. Combining simulations and real-robot experiments for bayesian optimization of bipedal gait stabilization. In *Robot World Cup*, 70–82 (Springer, 2018).
58. Gärtner, T. et al. A survey of kernels for structured data. *ACM SIGKDD Explor. Newsl.* **5**, 49–58 (2003).
59. Ontañón, S. An overview of distance and similarity functions for structured data. *Artif. Intell. Rev.* **53**, 5309–5351 (2020).
60. Kusner, M. J., Paige, B. & Hernández-Lobato, J. M. *Grammar Variational Autoencoder*. In *International Conference On Machine Learning*, 1945–1954 (PMLR, 2017).
61. Gómez-Bombarelli, R. et al. Automatic chemical design using a data-driven continuous representation of molecules. *ACS Cent. Sci.* **4**, 268–276 (2018).
62. Luo, R., Tian, F., Qin, T., Chen, E. & Liu, T.-Y. Neural architecture optimization. *Adv. Neural Inf. Process. Syst.* **31**, 7827–7838 (2018).
63. Tripp, A., Daxberger, E. & Hernández-Lobato, J. M. Sample-efficient optimization in the latent space of deep generative models via weighted retraining. *Adv. Neural Inf. Process. Syst.* **33**, 11259–11272 (2020).
64. Deshwal, A. & Doppa, J. Combining latent space and structured kernels for Bayesian optimization over combinatorial spaces. *Adv. Neural Inf. Process. Syst.* **34**, 8185–8200 (2021).
65. Borji, A. & Itti, L. Bayesian optimization explains human active search. *Adv. Neural Inf. Process. Syst.* **26** (2013).
66. Bishop, C. M. et al. *Pattern Recognition And Machine Learning*. (Springer, 2006).
67. Haussler, D. et al. Convolution kernels on discrete structures. *Tech. Rep. UC Santa Cruz* (1999).
68. Kondor, R. I. & Lafferty, J. Diffusion kernels on graphs and other discrete structures. In *Proceedings of the 19th International Conference on Machine Learning*. **2002**, 315–322 (2002).
69. Xu, K. et al. A bayesian-symbolic approach to reasoning and learning in intuitive physics. *Adv. Neural Inf. Process. Syst.* **34**, 2478–2490 (2021).
70. Yu, D., Yang, B., Liu, D., Wang, H. & Pan, S. A survey on neural-symbolic learning systems. *Neural Networks* (2023).
71. Tompkins, A. & Ramos, F. Fourier feature approximations for periodic kernels in time-series modelling. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32 (2018).
72. Mutny, M. & Krause, A. Efficient high dimensional bayesian optimization with additivity and quadrature fourier features. *Advances in Neural Information Processing Systems* 31 9005–9016 (2019).
73. Choromanski, K. M. et al. Hybrid random features. In *International Conference on Learning Representations* (2021).
74. Daniely, A., Frostig, R., Gupta, V. & Singer, Y. Random features for compositional kernels. *arXiv preprint arXiv:1703.07872* (2017).
75. Gärtner, T., Lloyd, J. W. & Flach, P. A. Kernels and distances for structured data. *Mach. Learn.* **57**, 205–232 (2004).
76. Kleyko, D., Rachkovskij, D. A., Osipov, E. & Rahimi, A. A survey on hyperdimensional computing aka vector symbolic architectures, part i: Models and data transformations. *ACM Computing Surveys (CSUR)* (2021).
77. Kleyko, D., Rachkovskij, D. A., Osipov, E. & Rahim, A. A survey on hyperdimensional computing, aka vector symbolic architectures, part ii: Applications, cognitive models, and challenges. *arXiv preprint arXiv:2112.15424* (2021).
78. Kleyko, D. et al. Vector symbolic architectures as a computing framework for nanoscale hardware. *arXiv preprint arXiv:2106.05268* (2021).
79. Schlegel, K., Neubert, P. & Protzel, P. A comparison of vector symbolic architectures. *Artif. Intell. Rev.* **55**, 4523–4555 (2022).
80. Orchard, J., Furlong, P. M. & Simone, K. Efficient hyperdimensional computing with spiking phasors. *Neural Comput.* **36**, 1886–1911 (2024).
81. Osipov, E. et al. Hyperseed: unsupervised learning with vector symbolic architectures. *arXiv preprint arXiv:2110.08343* (2021).
82. Haputhanthri, D. et al. Evaluating complex sparse representation of hypervectors for unsupervised machine learning. In *2022 International Joint Conference on Neural Networks (IJCNN)*, 1–6 (IEEE, 2022).
83. Plate, T. A. Holographic reduced representations. *IEEE Trans. Neural Netw.* **6**, 623–641 (1995).
84. Hannagan, T., Dupoux, E. & Christophe, A. Holographic string encoding. *Cogn. Sci.* **35**, 79–118 (2011).
85. Kleyko, D. et al. Vector symbolic architectures as a computing framework for emerging hardware. *Proc. IEEE* **110**, 1538–1571 (2022).

86. Gayler, R. W. & Levy, S. D. A Distributed Basis For Analogical Mapping. In *New Frontiers in Analogy Research; Proc. of 2nd Intern. Analogy Conf.* **9** (2009).
87. Laube, R. & Eliasmith, C. Qavsa: Question answering using vector symbolic algebras. In *Proceedings of the 9th Workshop on Representation Learning for NLP (Repl4NLP-2024)*, 191–202 (2024).
88. Sorscher, B., Mel, G., Ganguli, S. & Ocko, S. A unified theory for the origin of grid cells through the lens of pattern formation. In *Advances in Neural Information Processing Systems* (2019).
89. Komer, B. et al. *Biologically Inspired Spatial Representation*. Ph.D. thesis, University of Waterloo (2020).
90. Voelker, A. R. et al. Simulating and predicting dynamical systems with spatial semantic pointers. *Neural Comput.* **33**, 2033–2067 (2021).
91. Robbins, H. Some aspects of the sequential design of experiments. *Bull. Am. Math. Soc.* **58**, 527 – 535 (1952).
92. Lai, T. & Robbins, H. Asymptotically efficient adaptive allocation rules. *Adv. Appl. Math.* **6**, 4–22 (1985).
93. Auer, P., Cesa-Bianchi, N. & Fischer, P. Finite-time analysis of the multiarmed bandit problem. *Mach. Learn.* **47**, 235–256 (2002).
94. Srinivas, N., Krause, A., Kakade, S. M. & Seeger, M. *Gaussian Process Optimization in the Bandit Setting: No Regret and Experimental Design*. *Proceedings of the 27th International Conference on Machine Learning (ICML)* (2010).
95. Furlong, P. M., Stewart, T. C. & Eliasmith, C. Fractional binding in vector symbolic representations for efficient mutual information exploration. In *ICRA Workshop: Towards Curious Robots: Modern Approaches for Intrinsically-Motivated Intelligent Behavior*. 155 (2022).
96. Oh, C., Tomczak, J., Gavves, E. & Welling, M. Combinatorial Bayesian optimization using the graph cartesian product. *Advances in Neural Information Processing Systems* **32**, 2914–2924 (2019).
97. Merleau, N. S. & Smerlak, M. A simple evolutionary algorithm guided by local mutations for an efficient rna design. In *Proceedings of the genetic and evolutionary computation conference*, 1027–1034 (2021).
98. Dreczkowski, K., Grosnit, A. & Bou Ammar, H. Framework and benchmarks for combinatorial and mixed-variable bayesian optimization. *Adv. Neural Inf. Process. Syst.* **36**, 69464–69489 (2023).
99. Grosnit, A. et al. Boils: Bayesian optimisation for logic synthesis. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 1193–1196 (IEEE, 2022).
100. Deshwal, A. et al. Bayesian optimization over high-dimensional combinatorial spaces via dictionary-based embeddings. In *International Conference on Artificial Intelligence and Statistics*, 7021–7039 (PMLR, 2023).
101. Moss, H., Leslie, D., Beck, D., Gonzalez, J. & Rayson, P. Boss: Bayesian optimization over string spaces. *Adv. neural Inf. Process. Syst.* **33**, 15476–15486 (2020).
102. Wan, X. et al. Think global and act local: Bayesian optimisation over high-dimensional categorical and mixed search spaces. *arXiv preprint arXiv:2102.07188* (2021).
103. Ying, C. et al. Nas-bench-101: Towards reproducible neural architecture search. In *International conference on machine learning*, 7105–7114 (PMLR, 2019).
104. Kandasamy, K., Neiswanger, W., Schneider, J., Póczos, B. & Xing, E. P. Neural architecture search with Bayesian optimisation and optimal transport. *Adv. Neural Inf. Process. Syst.* **31** (2018).
105. White, C., Neiswanger, W. & Savani, Y. Bananas: Bayesian optimization with neural architectures for neural architecture search. In *Proceedings of the AAAI conference on artificial intelligence*, vol. 35, 10293–10301 (2021).
106. Shi, H. et al. Bridging the gap between sample-based and one-shot neural architecture search with bonas. *Adv. Neural Inf. Process. Syst.* **33**, 1808–1819 (2020).
107. Lukasik, J., Jung, S. & Keuper, M. Learning where to look—generative nas is surprisingly efficient. In *European Conference on Computer Vision*, 257–273 (Springer, 2022).
108. Asthana, R., Conrad, J., Dawoud, Y., Ortmanns, M. & Belagiannis, V. Multi-conditioned graph diffusion for neural architecture search. *arXiv preprint arXiv:2403.06020* (2024).
109. Daulton, S. et al. Bayesian optimization over discrete and mixed spaces via probabilistic reparameterization. *arXiv preprint arXiv:2210.10199* (2022).
110. Wesel, F. & Batselier, K. Large-scale learning with Fourier features and tensor decompositions. *Adv. Neural Inf. Process. Syst.* **34**, 17543–17554 (2021).
111. Dumont, N. S.-Y., Orchard, J. & Eliasmith, C. A model of path integration that connects neural and symbolic representation. In *Proceedings of the Annual Meeting of the Cognitive Science Society*, 44 (2022).
112. Dumont, N. S.-Y., Furlong, P. M., Orchard, J. & Eliasmith, C. Exploiting semantic information in a spiking neural slam system. *Front. Neurosci.* **17**, 1190515 (2023).
113. Snyder, S., Risbud, S. R. & Parsa, M. Neuromorphic bayesian optimization in lava. In *Proceedings of the 2023 International Conference on Neuromorphic Systems*, 1–5 (2023).
114. Snyder, S., Gobin, D., Clerico, V., Risbud, S. R. & Parsa, M. Parallelized multi-agent bayesian optimization in lava. In *2024 International Conference on Neuromorphic Systems (ICONS)*, 295–298 (IEEE, 2024).
115. Furlong, P. M., Dumont, N. S.-Y. & Orchard, J. A recurrent dynamic model for efficient bayesian optimization. In *2024 Neuro Inspired Computational Elements Conference (NICE)*, 1–5 (IEEE, 2024).
116. Doukas, M., Xydis, S. & Soudris, D. Dataflow acceleration of scikit-learn Gaussian process regression. In *Proceedings of the 8th Workshop and 6th Workshop on Parallel Programming and Run-Time Management Techniques for Many-core Architectures and Design Tools and Architectures for Multicore Embedded Computing Platforms*, 1–6 (2017).
117. Suzuki, H., Tsutsumi, S. & Sato, Y. An fpga implementation of a gaussian process based predictor for sequential time series data. In *2020 Eighth International Symposium on Computing and Networking Workshops (CANDARW)*, 445–449 (IEEE, 2020).
118. Deng, C. et al. Vlsi hardware architecture for gaussian process. In *2020 54th Asilomar Conference on Signals, Systems, and Computers*, 121–124 (IEEE, 2020).
119. Heddes, M., Nunes, I., Givargis, T., Nicolau, A. & Veidenbaum, A. Hyperdimensional hashing: A robust and efficient dynamic hash table. *arXiv preprint arXiv:2205.07850* (2022).
120. Jones, M. N. & Mewhort, D. J. Representing word meaning and order information in a composite holographic lexicon. *Psychol. Rev.* **114**, 1 (2007).
121. Graben, P. B., Huber, M., Meyer, W., Römer, R. & Wolff, M. Vector symbolic architectures for context-free grammars. *Cognitive Computation* 1–16 (2022).
122. Penzkofer, A., Shi, L. & Bulling, A. Vsa4vqa: Scaling a vector symbolic architecture to visual question answering on natural images. In *Proc. 46th Annual Meeting of the Cognitive Science Society (CogSci)* (2024).
123. Krupic, J., Bauza, M., Burton, S. & O’Keefe, J. Local transformations of the hippocampal cognitive map. *Science* **359**, 1143–1146 (2018).
124. Dupret, D., O’neill, J., Pleydell-Bouverie, B. & Csicsvari, J. The reorganization and reactivation of hippocampal maps predict spatial memory performance. *Nat. Neurosci.* **13**, 995–1002 (2010).
125. Krichmar, J. L., Nitz, D. A., Gally, J. A. & Edelman, G. M. Characterizing functional hippocampal pathways in a brain-based

- device as it solves a spatial memory task. *Proc. Natl. Acad. Sci.* **102**, 2111–2116 (2005).
126. Kudithipudi, D. et al. Biological underpinnings for lifelong learning machines. *Nat. Mach. Intell.* **4**, 196–210 (2022).
  127. Moser, E. I. & Moser, M.-B. A metric for space. *Hippocampus* **18**, 1142–1156 (2008).
  128. Gosmann, J. & Eliasmith, C. Optimizing semantic pointer representations for symbol-like processing in spiking neural networks. *PLoS One* **11**, e0149928 (2016).
  129. Voelker, A. R., Gosmann, J. & Stewart, T. C. Efficiently sampling vectors and coordinates from the n-sphere and n-ball. *Centre for Theoretical Neurosci. Tech. Rep.* **1** (2017).
  130. Frady, E. P., Kent, S. J., Olshausen, B. A. & Sommer, F. T. Resonator networks, 1: An efficient solution for factoring high-dimensional, distributed representations of data structures. *Neural Comput.* **32**, 2311–2331 (2020).
  131. Stewart, T., Choo, F.-X. & Eliasmith, C. *Sentence Processing In Spiking Neurons: A Biologically Plausible Left-corner Parser*. In *Proceedings of the Annual Meeting of the Cognitive Science Society*, **36** (2014).
  132. Renner, A. et al. Neuromorphic visual scene understanding with resonator networks. *Nat. Mach. Intell.* **6**, 641–652 (2024).
  133. Plebe, A., Pavone, A. et al. *Neural Semantic Pointers in Context*. In *IJCCI*, 447–454 (2020).
  134. Komer, B. & Eliasmith, C. Efficient navigation using a scalable, biologically inspired spatial representation. In *42nd Annual Meeting of the Cognitive Science Society* (Cognitive Science Society, Toronto, ON, 2020).
  135. Gosmann, J. & Eliasmith, C. Vector-derived transformation binding: An improved binding operation for deep symbol-like processing in neural networks. *Neural Comput.* **31**, 849–869 (2019).
  136. Sriperumbudur, B. & Szabo, Z. Optimal rates for random Fourier features. In Cortes, C., Lawrence, N., Lee, D., Sugiyama, M. & Garnett, R. (eds.) *Advances in Neural Information Processing Systems*, **28** (Curran Associates, Inc. [https://proceedings.neurips.cc/paper\\_files/paper/2015/file/d14220ee66aeec73c49038385428ec4c-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2015/file/d14220ee66aeec73c49038385428ec4c-Paper.pdf)). (2015).
  137. Bach, F. et al. *Sharp Analysis of Low-rank Kernel Matrix Approximations*. In *Conference on Learning Theory*, 185–209 (PMLR, 2013).
  138. Li, Z., Ton, J.-F., Oglic, D. & Sejdinovic, D. *Towards a unified analysis of random Fourier features*. In *International conference on machine learning*, 3905–3914 (PMLR, 2019).
  139. Rudi, A. & Rosasco, L. Generalization properties of learning with random features. *Advances in neural information processing systems*. **30**, 3218–3228 (2017).
  140. Eliasmith, C. & Anderson, C. H. *Neural Engineering: Computation, Representation, And Dynamics In Neurobiological Systems* (MIT Press, 2003).
  141. Buitinck, L. et al. API design for machine learning software: experiences from the scikit-learn project. In *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 108–122 (2013).
  142. Osthege, M. & Felton, K. Michaelosthege/pyrff: v2.0.3 <https://doi.org/10.5281/zenodo.15048204>. (2020).
  143. Balandat, M. et al. *BoTorch: A Framework for Efficient Monte-Carlo Bayesian Optimization*. In *Advances in Neural Information Processing Systems* <http://arxiv.org/abs/1910.06403> (2020).
  144. Gardner, J., Pleiss, G., Weinberger, K. Q., Bindel, D. & Wilson, A. G. Gpytorch: Blackbox matrix-matrix gaussian process inference with gpu acceleration. *Adv. Neural Inf. Process. Syst.* **31** (2018).
  145. Shahriari, B., Swersky, K., Wang, Z., Adams, R. & De Freitas, N. Taking the human out of the loop: a review of Bayesian optimization. *Proc. IEEE* **104**, 148–175 (2016).
  146. Ament, S. et al. Robust Gaussian processes via relevance pursuit. *Adv. Neural Inf. Process. Syst.* **37**, 61700–61734 (2024).
  147. Maus, N. et al. *Approximation-aware Bayesian optimization*. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems* (2024).
  148. Buathong, P. et al. Bayesian optimization of function networks with partial evaluations. In Salakhutdinov, R. et al. (eds.) *Proceedings of the 41st International Conference on Machine Learning*, vol.235 of *Proceedings of Machine Learning Research*, 4752–4784 (PMLR, 2024).
  149. Lizotte, D. J., Wang, T., Bowling, M. H. & Schuurmans, D. *Automatic Gait Optimization with Gaussian Process Regression*. In *International Joint Conference on Artificial Intelligence (IJCAI)*, **7**, 944–949 (2007).
  150. Tesch, M., Schneider, J. & Choset, H. *Using Response Surfaces And Expected Improvement To Optimize Snake Robot Gait Parameters*. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 1069–1074 (IEEE, 2011).
  151. Caccamo, S., Bekiroglu, Y., Ek, C. H. & Kragic, D. Active exploration using Gaussian random fields and gaussian process implicit surfaces. In *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 582–589 (IEEE, 2016).
  152. Calandra, R., Seyfarth, A., Peters, J. & Deisenroth, M. P. Bayesian optimization for learning gaits under uncertainty. *Ann. Math. Artif. Intell.* **76**, 5–23 (2016).
  153. Antonova, R., Rai, A., Li, T. & Kragic, D. Bayesian optimization in variational latent spaces with dynamic compression. In *Conference on Robot Learning (CoRL)* (2019).
  154. Demir, S. O. et al. Task space adaptation via the learning of gait controllers of magnetic soft millirobots. *Int. J. Robot. Res.* **40**, 1331–1351 (2021).
  155. Hensman, J., Fusi, N. & Lawrence, N. D. Gaussian processes for big data. *arXiv preprint arXiv:1309.6835* (2013).
  156. Burt, D. R., Rasmussen, C. E. & Van Der Wilk, M. Convergence of sparse variational inference in Gaussian processes regression. *J. Mach. Learn. Res.* **21**, 1–63 (2020).
  157. Coumans, E. & Bai, Y. PyBullet, a Python module for physics simulation for games, robotics and machine learning. <http://pybullet.org> (2016–2022).
  158. Furlong, P. M. et al. Foraging, prospecting, and falsification-improving three aspects of autonomous science. *Tech. Rep.* Carnegie Mellon University (2018).

## Acknowledgements

We would like to thank Paxon Frady, Danielle Rager, Sumedh Risbud, and Alessandro Pierro for their advice and assistance with Loihi 2 experiments. Discussions with Mr. Pierro occurred at the 2025 Telluride Neuromorphic Cognition Engineering Workshop. PMF would like to thank Dr. Maddy Bartlett for useful conversations.

## Author contributions

P.M.F., N.S.-Y.D., and R.A. contributed equally to the development of the algorithms, the design of experiments, the creation of software, and the writing of this article. J.O. contributed to the design of experiments and the writing of the article. C.E. contributed to the theoretical development of the methods, design of the experiments and the writing of the article.

## Funding

P. M. Furlong, C. Eliasmith, and N. S.-Y. Dumont were funded in part by CFI and OIT infrastructure funding, Canada Research Chairs program, NSERC Discovery grant 261453, NUCC NRC File A-0028850, AFOSR grant FA9550-23-1-0644. This research was enabled by support and resources from the Digital Resource Alliance of Canada. P.M. Furlong, N. S.-Y. Dumont, and J. Orchard were funded by an Intel Neuromorphic

Research Community Grant. R. Antonova was supported by the National Science Foundation grant No.2030859 to the Computing Research Association for the CIFellows Project.

## Competing interests

Chris Elias Smith has a financial interest in Applied Brain Research, Incorporated, holder of patents related to the material in this paper (patents 12,014,263 and 17/520,379). The company or this cooperation did not affect the authenticity and objectivity of the experimental results of this work. The funders had no role in the direction of this research; in the analyses or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

## Additional information

**Supplementary information** The online version contains supplementary material available at <https://doi.org/10.1038/s41467-026-75703-4>.

**Correspondence** and requests for materials should be addressed to P. Michael Furlong.

**Peer review information** *Nature Communications*, thanks Michael Hersche, Ross Gayler and the other, anonymous, reviewer(s) for their contribution to the peer review of this work. A peer review file is available.

**Reprints and permissions information** is available at <http://www.nature.com/reprints>

**Publisher's note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

**Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

© Crown 2026

This Article in Press is shared early to give you faster access to new research. It is citable and carries a permanent DOI. The final edited version will replace it automatically.